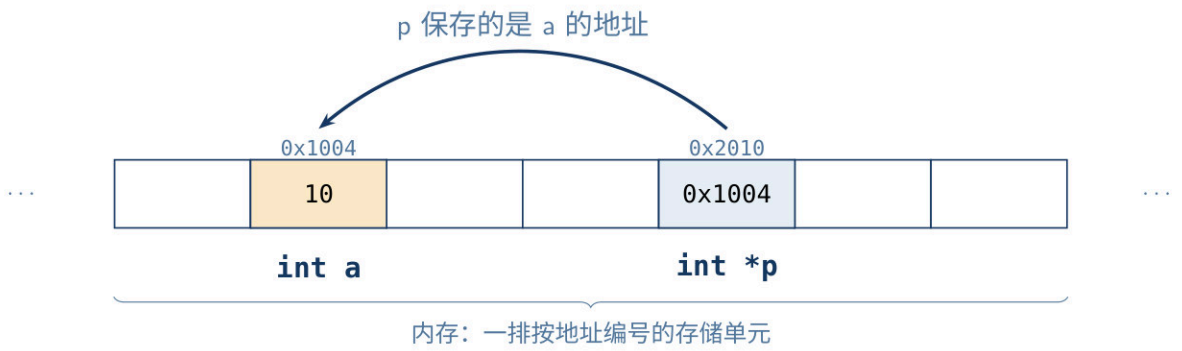
C++ 程序设计 · 第七章

指 针

从“内存地址”到“用指针排序数组”

基础学习讲义 可视化图解 · 推导补全 · 分层练习



适用对象	第一次学习 C++ 指针、前置基础不太牢固的本科生
原始依据	针与野指针、const 修饰指针、指针和数组、指针和函数、指针 + 数组 + 函数综合案例）
本讲义做了什么	38 道分层练习与逐选项图解
编译环境	XeLaTeX；全部示例代码已用 g++（C++17，64 位 Linux）实际编译运行

阅读说明

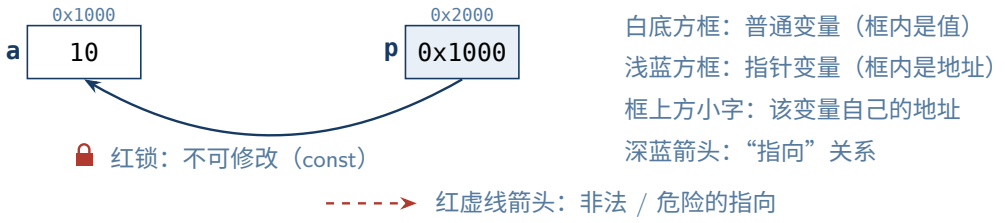
这份讲义不是把笔记“抄一遍”，而是把笔记里讲到的每一个知识点，按照“先知道为什么、再建立图像、然后给定义和公式、最后练习”的顺序重新讲一遍。建议这样使用：

第一次学习	按顺序读第一讲～第八讲，每讲末尾的“自检”能答出来再往后走。遇到内存图，先用手指沿着箭头“走一遍”。
课前预习	只读每讲开头的“本讲回答的问题”和“直观理解”框，建立印象即可。
课后复习	重点看“易错点”（浅红色框）和“相近概念辨析”表格，再做 A 组基础题。
考前巩固	直接做练习 A、B、C 三组，对照答案速查，错题再回到对应讲次（见“知识点—题目对应表”）。

颜色约定 全书颜色含义固定，读几页之后就能“看颜色知用途”：

定义·核心知识　核心关系式　例题·历史·科普　易错·原图纠错　总结·自检

内存图约定 本书所有内存图使用同一套画法，请先认识它们：



提示 图中的地址（如 `0x1000`）都是为了讲解而选的“好看的数”。真实程序每次运行打印出的地址一般不同，且通常很长（如 `0x7ffcc6ee782c`）。

目 录

阅读说明 1

导读：本专题学什么、在哪里、需要什么 3

学科小史：指针是怎样出现的？ 4

从课本到现实：指针到底有什么用？ 5

原始笔记的识别、重编码与纠错 6

第一讲 内存与地址：指针要保存的到底是什么 9

第二讲 指针变量的定义与使用：& 与 * 12

第三讲 指针所占的内存空间：sizeof(p) 15

第四讲 空指针与野指针：哪些地址不能碰 17

第五讲 const 修饰指针：给指针“上锁” 20

第六讲 指针和数组：让指针在数组上移动 23

第七讲 指针和函数：值传递与地址传递 26

第八讲 指针、数组和函数：封装冒泡排序 30

本章知识地图与总自检 35

知识点—题目对应表 36

正式练习 37

全部答案速查 51

A 组详细解析 52

B 组详细解析 62

C 组详细解析（跨学科迁移） 67

高频易错点总表 69

基础解题步骤总结 70

相近概念总对比 70

后续课程连接 71

参考资料 71

导读：本专题学什么、在哪里、需要什么

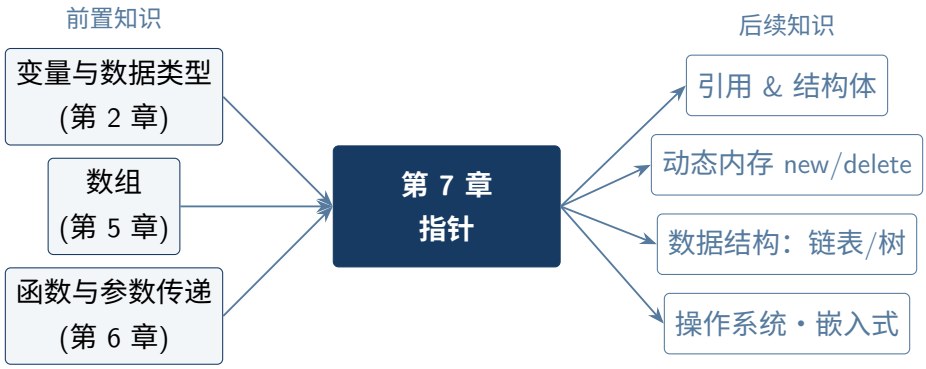
本专题回答的核心问题

学到第七章之前，我们一直在用“变量名”操作数据：`a = 10;`、`cout << a;`。这很方便，但留下了几个解决不了的问题：

1. 一个函数想**修改调用者的变量**（比如交换两个数），只拿到“值的副本”是做不到的；
2. 把一个很大的数组交给函数处理时，我们不希望把整个数组复制一份，只想告诉函数**“数据在哪儿”**；
3. 有些数据根本没有名字，只有位置（例如以后学的动态内存、硬件寄存器），只能按**地址**访问。

这三个问题的共同答案就是：**把“地址”本身也当成一种数据，存起来、传出去、用它间接访问内存**。存放地址的变量就叫**指针**。

本专题在课程体系中的位置



前置知识地图：开始之前你需要知道的五件事

前置知识	本章要用到的最少内容	用在哪里
变量与类型	`int a = 10;` 会在内存中划出一块空间（`int` 一般 4 字节）并存入 10	第一讲、第二讲
sizeof	`sizeof(x)` 给出 `x` 占用的字节数，编译时就已确定	第三讲、第八讲
数组	`int arr[5]` 是 5 个 `int` **紧挨着**存放；下标从 0 开始，合法范围 `0`～`4`	第六讲、第八讲
函数	调用 `f(a)` 时，形参是实参的**拷贝**（值传递）	第七讲
十六进制	`0x` 开头表示十六进制：`0x10`= 16，`0x0C`= 12	全章地址计算

> **教学补全：十六进制快速复习**
>
> 原始笔记未完整展示该部分，以下按照本课程常见教材与 C++ 标准表述补充。

十六进制每一位取 `0`〜`9`，`A`〜`F`（A=10，…，F=15），第 k 位（从右往左、从 0 数）的权是 16^k。例如

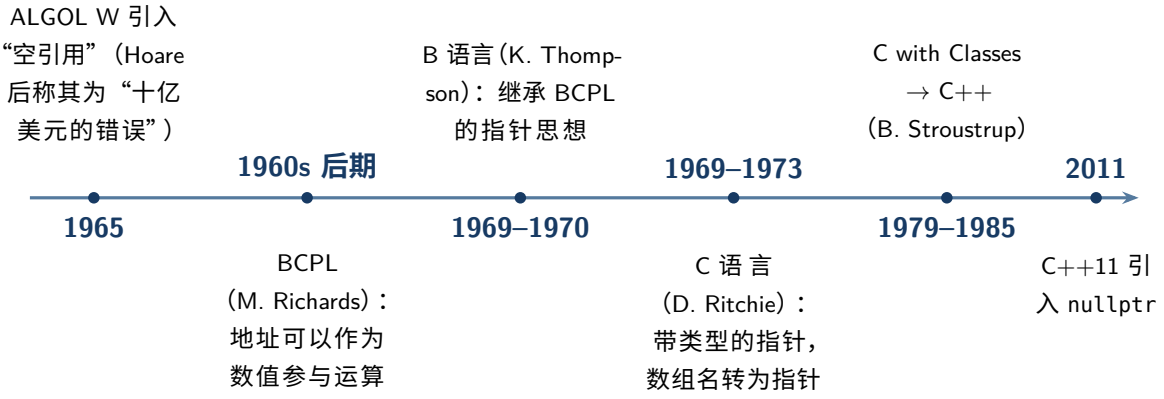
$$\texttt{0x0010} = 1 \times 16^1 + 0 = 16, \qquad \texttt{0x100C} = 1 \times 16^3 + 0 + 0 + 12 = 4108.$$

地址习惯用十六进制书写，原因是一位十六进制恰好对应 4 个二进制位，两位恰好是 1 个字节，和计算机“按字节编址”的方式对得很整齐。计算地址时最常见的操作是“加几个字节”，例如 `0x1000` + 12 = `0x100C`（因为十进制 12 就是十六进制 C）。

学科小史：指针是怎样出现的?

今天我们觉得“变量有地址”是理所当然的，但在高级语言发展早期，这是一个需要取舍的设计问题：**语言应该把机器的“地址”暴露给程序员，还是把它藏起来?**

汇编语言里程序员直接写地址，效率高但极易出错；许多早期高级语言则把地址藏起来，安全但难以编写操作系统这类需要直接操纵内存的程序。指针正是在“既要像汇编一样高效，又要像高级语言一样可读”的需求下成熟起来的。



几个真正影响你今天写代码的节点

- **BCPL 与 B（1960 年代后期—1970 年）**：Martin Richards 设计的 BCPL 和 Ken Thompson 在它基础上做的 B 语言，都允许把地址当作数来运算。但它们只有“机器字”一种数据，指针加 1 就是“下一个字”，没有“类型”的概念。
- **C 语言（1969—1973，贝尔实验室）**：Dennis Ritchie 在 B 的基础上加入了 **`int`**、**`char`** 等类型，指针也随之有了类型——**`int`*** 加 1 前进一个 **`int`** 的字节数，**`char`*** 加 1 前进 1 个字节。Ritchie 在回顾文章中特别提到，“数组名在表达式中转换为指向首元素的指针”这一规则，是从 B 平滑过渡到 C 的关键设计。这正是本讲义第六讲的核心。
- **C++（1979 年起）**：Bjarne Stroustrup 从 1979 年开始在 C 上增加类，最初叫“C with Classes”，1983 年改名 C++，1985 年对外发布。C++ 完整保留了 C 的指针，所以本章内容在 C 和 C++ 中几乎一致。

- **空指针的反思**：Tony Hoare 在 2009 年的一次演讲中回顾，他在 1965 年设计 ALGOL W 时引入了“空引用”，并称之为自己的“十亿美元的错误”——因为它引发了无数程序崩溃。C++11 引入更安全的 `nullptr`，现代语言则进一步从类型系统上限制空值。这正是第四讲“空指针不可访问”背后的历史教训。

读完这段历史，你会发现：**指针的“类型”决定步长、数组名会变成指针、空指针不能解引用**——这三条今天考试最爱考的规则，都是语言设计者当年有意做出的选择。

从课本到现实：指针到底有什么用？

1. 操作系统：你见过的“段错误”是怎么来的

笔记里写“空指针访问 → segmentation fault（段错误）”。这背后是操作系统的**内存保护**：每个程序只能访问操作系统分配给它的那部分地址；主流操作系统通常故意不把地址 0 附近的区域分配给程序。当程序通过空指针访问地址 0 时，硬件发现“这个地址不属于你”，操作系统就把程序终止，并报告段错误。所以段错误其实是一种**保护**：宁可让程序立刻崩溃，也不让它悄悄改坏别人的数据。

2. 嵌入式：用指针“按地址”点亮一盏灯

在单片机里，控制 LED、电机的“寄存器”被安排在固定地址上（由芯片手册规定）。程序员把这个地址强制转换成指针，然后 `*reg = 1;` 就能让引脚输出高电平。这和笔记里 `int *p = (int*)0x0010;` 的写法**形式完全相同**——区别只在于：单片机上这个地址是芯片手册保证可以访问的，而在电脑上随便写的地址不是你的，就成了野指针。（C 组第 37 题就考这个。）

3. 图像与数据：一张照片就是一个很长的数组

一张 1920×1080 的灰度图片，在内存中通常就是 1920×1080 个字节**按行连续存放**。图像处理程序拿到一个指向首字节的指针，用“指针 + 偏移”走遍每个像素；把图片交给函数处理时，也只传指针和尺寸，而不复制几兆字节的数据——这就是第八讲“数组作函数参数时只传首地址”的现实版本。（C 组第 38 题。）

4. 安全：为什么大家越来越重视“内存安全”

笔记中“数组 5 个元素却循环 10 次”的越界读取，在真实软件里是一类非常常见、危害很大的安全漏洞来源（缓冲区溢出）。这也是近年来越来越多的工程团队重视内存安全、并在部分场景采用带边界检查的容器或内存安全语言的原因。学好指针的**边界意识**，是写出安全程序的第一步。

原始笔记的识别、重编码与纠错

原始资料内容一览

原始资料为手写笔记照片 11 页（笔记页码 378–393，日期 2026 年 9 月 25–27 日）。按照笔记本身的小节，识别出的结构如下：

笔记小节	标题	可靠识别的主要内容
§7.1	基本概念	作用：通过指针间接访问内存；内存编号从 0 开始，一般用十六进制表示；可以利用指针变量保存地址；图示 `int a = 10`、`p` 中存 `0x0000`（红字：“可以通过指针保存一个地址”）
§7.2	指针变量的定义与使用	语法“数据类型 * 指针变量名”；`int *p; p = &a;`；输出 `p`；“可以通过解引用的方式来找到指针指向的内存”，`*p`（红字标注“解引用”）
§7.3	指针所占的内存空间	32 位操作系统 4 字节，64 位操作系统 8 字节；示例 `int b = sizeof(p);`
§7.4	空指针与野指针	空指针：指向编号为 0 的空间，用于初始化，不可访问，`int *p = NULL; *p = 100` → segmentation fault；野指针：指向非法的内存空间，示例 `int *p = (int*)0x0010;`；红字总结：“空指针和野指针都不是我们申请的空间，因此不要访问”
§7.5	const 修饰指针	三种情况：const 修饰指针——常量指针（`const int *p`，指向可改、值不可改）；const 修饰常量——指针常量（`int * const p`，指向不可改、值可改）；两者都修饰（`const int * const p`）；图示与示例代码
§7.6	指针和数组	利用指针访问数组元素；`int *p = arr;`，`p++`（红字：“向后偏移 4 个字节”）；通过指针遍历数组
§7.7	指针和函数	利用指针作函数参数可以修改实参；回顾值传递 `swap(int a,int b)`；地址传递 `swap_p(int *p1,int *p2)`，红字“可以改变实参”

笔记小节	标题	可靠识别的主要内容
§7.8	指针、数组和函数	案例：封装函数，利用冒泡排序实现整型数组升序排序；先在 `main` 中写冒泡排序并解释两层循环边界；再封装 `BubbleSort(int *arr,int n)`、`PrintArray`，在 `main` 中用 `sizeof(arr)/sizeof(arr[0])` 求长度

无法可靠辨认 / 需要说明的内容

- 第 378 页“一般用十_ 进制数字表示”中间一字书写潦草，**原始资料此处无法完全可靠辨认**。*可能解释：“十六进制”*——依据是同页地址均写作 `0x...` 形式。以下属于推测，不属于原图可靠识别结果，讲义按“十六进制”讲解。
- 第 391 页 `int arr[] = [5, 3, 7, 8, 2}` 的左括号形似“[”。C++ 中数组初始化只能用花括号，讲义按 `{5,3,7,8,2}` 处理。
- 第 389 页 `cout << "a=" << a < " b=" ...` 中 `a` 后面似乎只写了一个 `<`。可能是书写时漏笔，讲义按 `<<` 处理。
- 部分页面背面字迹透印（如第 380、388 页中的淡色文字），均为其他页内容的透印，未计入本页识别结果。

原图纠错

以下 6 处问题都经过 g++ 实际编译或运行验证。它们不是“挑刺”，每一处都对应一个初学者很容易踩的坑，所以后文的讲解和练习会反复回到这里。

原图纠错 · ① §7.6 遍历数组的循环次数越界（第 388 页）

原始写法　`int arr[] = {1,2,3,4,5};` 之后 `for (int i = 0; i < 10; i++) { cout << *p << endl; p++; }`

存在的问题　数组只有 5 个元素，循环却执行 10 次，后 5 次读取的是数组之外的内存，属于**越界访问（未定义行为）**。用 AddressSanitizer 编译运行，程序在第 6 次读取时报告 `stack-buffer-overflow`。

规范写法　`int len = sizeof(arr) / sizeof(arr[0]);  for (int i = 0; i < len; i++)`

为什么这样改　循环次数应由数组实际长度决定；用 `sizeof` 计算长度，数组改了元素个数也不会出错。　**影响**　第六讲、第 24 题专门讨论。

原图纠错 · ② §7.8 封装后的冒泡排序内层循环条件写错（第 392 页）

原始写法 `for (int j = 0; j < n - j - 1; j++)`
存在的问题 应为 `n - i - 1`。写成 `n - j - 1` 后，条件变成 $2j < n-1$，内层循环只比较数组的前一半。实测：对笔记中的 `{3,7,9,12,5,1}`，输出仍是 `3 7 9 12 5 1`，**完全没有排好**。（同一笔记第 391 页未封装的版本写的是正确的 `j < n-i-1`。）
规范写法 `for (int j = 0; j < n - i - 1; j++)`　　**影响** 第八讲推导为什么是 $n-i-1$；第 30、35 题。

原图纠错 · ③ §7.8 if 语句缺少括号（第 392 页）

原始写法 `if arr[j] > arr[j+1]`　　**问题** C++ 语法要求 `if` 的条件必须写在圆括号中，否则编译错误。　　**规范写法** `if (arr[j] > arr[j+1])`

原图纠错 · ④ §7.5 注释中的变量名写错（第 387 页）

原始写法 `const int * const p2 = &a; // p1 = &b 与 *p1 = b 均非法`
存在的问题 这一行讨论的是 `p2`；而 `p1` 是上面定义的 `const int *p1`，`p1 = &b` 恰恰是**合法的**。　**规范写法** `// p2 = &b 与 *p2 = b 均非法`

原图纠错 · ⑤ §7.5 图示中 b 的值画错（第 381 页）

原始写法 图中标注 `0x0022 → 20`、代码 `int b = 20`，但下方方框 `0x0022` 中写的是 `10`。
规范写法 地址 `0x0022` 的方框应为 `20`。图中指针标为 P_1 而代码写 `p`，讲义统一为代码中的名字。　**影响** 只影响图示，不影响结论。第五讲重新绘制。

原图纠错 · ⑥ §7.3 “32 位操作系统 4 字节 / 64 位操作系统 8 字节”表述不够严谨（第 379 页）

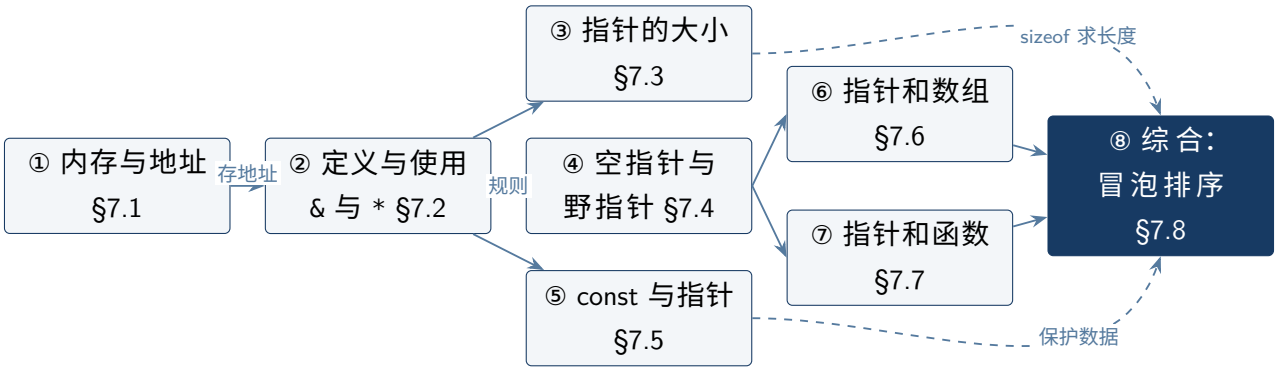
存在的问题 指针大小由**程序编译的目标平台**决定，而不是由操作系统直接决定。例如在 64 位 Windows 上运行的 32 位程序，`sizeof(int*)` 仍然是 4。
规范写法 “在 32 位程序（32 位目标平台）中，指针通常占 4 字节；在 64 位程序中，通常占 8 字节。”　**影响** 考试中按“32 位 4 字节、64 位 8 字节”作答即可，但要知道“位数”指程序的目标平台。

另外两处值得注意的说法（不算错误，但需补充）

- 笔记说“空指针访问 → segmentation fault”。严格说，C++ 标准规定解引用空指针是**未定义行为**；在 Linux、Windows 等主流系统上*通常*表现为段错误或访问冲突，但**不保证**一定崩溃。
- 笔记用 `0x0000` 作为变量 `a` 的示意地址。这只是示意；在真实系统中地址 0 正是空指针的值，普通变量不会位于那里。本讲义的示意地址统一从 `0x1000` 起。

知识点总览与关系图

笔记的八个小节并不是八个孤立的话题，而是一条“先有地址，再有指针，再用指针连接数组和函数”的链条。本讲义按下面的学习逻辑组织（与笔记顺序基本一致，只是把“内存与地址”单独拿出来讲透）：



实线箭头表示“后者必须以前者为基础”，虚线表示“后者会用到前者的某个细节”。整章的终点是第八讲：**数组（数据）通过指针（地址）交给函数（操作）去排序**——三者在一个例子里会合。

第一讲　内存与地址：指针要保存的到底是什么

本讲回答三个问题： ① 变量在内存里是什么样子？② “地址”是什么，为什么常用十六进制？③ 为什么要有一种专门保存地址的变量？

知识路线：内存按字节编号 → 变量占一段连续字节 → 变量的地址 = 首字节编号 → 把地址存进变量 = 指针

学科坐标：C++ 程序设计第 7 章 §7.1 ｜前置：变量、数据类型｜后续：指针定义、数组、函数传参｜相关课程：计算机组成原理、操作系统

1.1　它解决什么问题

笔记开篇第一句话是：“**作用：通过指针间接访问内存**”。“间接”是相对于“直接”而言的：

- **直接访问**：写 `a = 20;`，编译器知道 `a` 在哪儿，直接去改；
- **间接访问**：先拿到“`a` 在哪儿”这个信息（地址），再*按这个位置*去改。

为什么要绕这个弯？因为很多时候，写代码的地方**看不到变量的名字**。比如第七讲的交换函数：`swap` 函数内部根本不知道 `main` 里有个叫 `a` 的变量，但只要把 `a` 的*地址*交给它，它就能找到并修改 `a`。要理解这一点，先要弄清“地址”是什么。

1.2 先看直观图像：内存是一排带编号的小格子

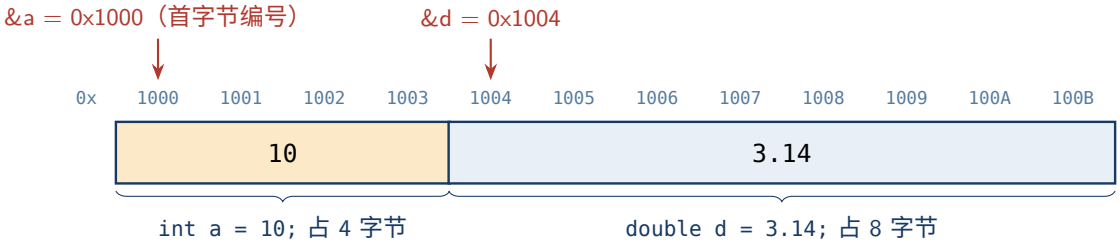


图 1.1　内存按字节编号；一个变量占用一段连续字节，变量的地址就是它第一个字节的编号

读图：横向每个小格是 **1 个字节**，格子上方的小字是它的编号（十六进制）。黄色大框是 `int` 变量 `a`，横跨 4 个格子；蓝色大框是 `double` 变量 `d`，横跨 8 个格子。红色箭头指出：**一个变量的地址只记它第一个字节的编号**。

1.3 正式定义

定义：内存地址

计算机内存以**字节**为基本单位，每个字节都有一个唯一的编号，这个编号称为该字节的**地址**。编号从 0 开始依次递增，习惯上用十六进制书写（前缀 `0x`）。
一个类型为 `T` 的变量在内存中占用 `sizeof(T)` 个**连续**字节；这段字节中**第一个字节的地址**称为该变量的地址，可以用取地址运算符 `&` 得到。

核心关系：变量占用的地址范围

$$\boxed{\text{变量}x\text{ 占用的地址} \;=\; \&x,\;\; \&x+1,\;\; \ldots,\;\; \&x+\texttt{sizeof}(x)-1}$$

$\&x$：变量 x 的首字节地址；$\texttt{sizeof}(x)$：x 占用的字节数（`int` 通常为 4，`double` 通常为 8，`char` 规定为 1）。
使用条件：这里的“$+1$”是**按字节**加的普通整数加法——这和第六讲“指针加 1”不同，届时会专门比较。

为什么地址习惯用十六进制

地址本质上只是一个整数，写成十进制也完全正确。但计算机内部用二进制，而 1 位十六进制正好是 4 位二进制，2 位十六进制正好是 1 个字节（8 位）。所以十六进制地址既比二进制短，又能一眼看出字节边界。笔记中“一般用十六进制数字表示”说的就是这件事——它是**书写习惯**，不是地址本身的性质。

例题 1.1　计算一个变量占用的地址范围

已知：`double d;` 的地址 `&d` 为 `0x1004`，`sizeof(double)` 为 8。　**求**：`d` 占用哪些地址？紧接着存放的下一个变量最早从哪里开始？
思路：套用“地址范围”关系，注意用十六进制做加法。

第一步：最后一个字节地址 $=$ `0x1004` $+ 8 - 1 =$ `0x1004` $+ 7 =$ `0x100B`（十进制 11 写作 B）。
第二步：`d` 占 `0x1004`～`0x100B`，所以下一个变量最早从 `0x100C` 开始。
检查：`0x100C`$-$`0x1004`$= 8$，恰好是 `double` 的大小。　**本题真正想说明**：变量是“一段”字节，而地址只记“起点”；长度由类型决定。对照图 1.1 的蓝框即可看到这 8 个格子。

1.4　把地址存起来：指针的想法

既然地址是一个数，它当然也可以**存进另一个变量里**。笔记中的图示正是这个意思（红字：“可以通过指针保存一个地址”）：

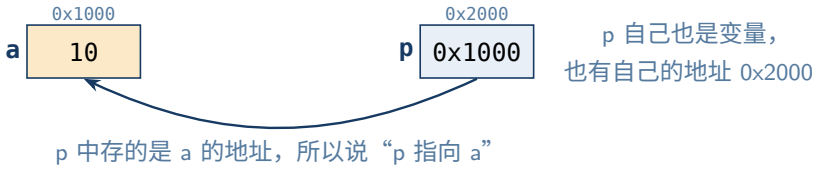


图 1.2　笔记 §7.1 图示的重绘：普通变量存“值”，指针变量存“地址”

这里有两个“地址”，一定要分清：`0x1000` 是 `a` 的地址，它被当作*值*存放在 `p` 里；`0x2000` 是 `p` *自己*的地址。箭头只是一种画法，它的含义就是“`p` 的值等于 `a` 的地址”。

易错点

1. **把“变量名”当作地址。**变量名只存在于源代码中，是给人看的标签；程序运行时使用的是地址。`a` 的地址要写成 `&a`。
2. **以为地址是固定的。**同一个程序每次运行，局部变量的地址通常都不一样（现代操作系统会随机化地址布局）。所以题目中不会问“`&a` 等于多少”，除非题目先给出示意地址。
3. **忘记变量占多个字节。**`int` 变量的地址是 `0x1000`，并不代表它只占 `0x1000` 这一个字节。

学完这里，你应该能够回答

1. 内存的基本编址单位是什么？一个 `int` 变量有几个字节、几个地址、但“它的地址”是哪一个？
2. `0x1000` 与 `0x1010` 相差多少字节？
3. 用一句话说明“直接访问”和“间接访问”的区别。
4. 图 1.2 中哪个数是 `a` 的地址？哪个数是 `p` 的地址？

到这里，我们已经解决了什么？　知道了地址就是字节编号，并且地址可以存进变量。**下一步还需要解决什么？**在 C++ 里怎样声明这种变量、怎样拿到一个变量的地址、又怎样“顺

着地址”去读写内存？这就是第二讲的 & 与 *。

第二讲　指针变量的定义与使用：& 与 *

本讲回答三个问题： ① 怎样定义一个指针变量？② 怎样让它记录某个变量的地址？③ 怎样通过它读、写那个变量？

知识路线：　定义 `int *p` → 取地址 `p = &a` → 解引用 `*p` 读写 → 三条恒等关系

2.1　正式定义

定义：指针变量

存放地址的变量称为**指针变量**（简称指针）。定义语法（笔记 §7.2）：

数据类型 * 指针变量名;　　例如　`int *p;`

其中“数据类型”说明这个指针**将要指向的变量的类型**：`int *p` 读作“p 是指向 int 的指针”，p 本身的类型写作 `int*`。

定义：取地址运算符 & 与解引用运算符 *

- `&x`：得到变量 x 的地址。若 x 的类型是 T，则 `&x` 的类型是 `T*`。
- `*p`：按照 p 中保存的地址找到那块内存，**把它当作一个 T 类型的变量**来使用——既可以读，也可以写。这个操作叫**解引用**。

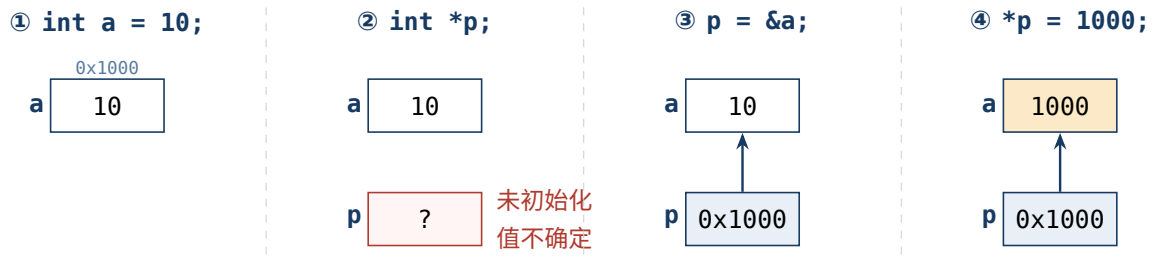


图 2.1　四行代码的内存变化：③ 建立“指向”关系，④ 顺着箭头修改的是 a 本身

图 2.1 按执行顺序画出了四个时刻。第 ② 步 p 刚定义时没有初值，框里是“?”——此刻它是一个**野指针**（第四讲详谈）；第 ③ 步 p 中存入 a 的地址，箭头出现；第 ④ 步 `*p = 1000` 顺着箭头找到 a，把 a 改成 1000，而 p 的值**没有变**。

2.2 核心关系与推导

核心关系：取地址与解引用互为“逆操作”

设 `T x;` 是一个变量，`T *p = &x;`，则

$$\boxed{*p \equiv x} \qquad *\&x \equiv x \qquad \&*p = p$$

这里“≡”表示“是同一个变量（同一块内存）”，而不仅仅是“值相等”。

使用条件：`p` 必须指向一个**有效的**对象。若 `p` 是空指针或野指针，`*p` 本身就是非法的，三条关系都无从谈起。

推导：为什么 *p 就是 x 本身

要证明：当 `p = &x` 时，对 `*p` 的读写就是对 `x` 的读写。

已知：(1) `&x` 的值是 `x` 的首字节地址，记为 A；(2) 赋值后 `p` 的值为 A（由赋值语句的含义）；(3) `*p` 的含义是“从 `p` 的值所表示的地址开始、按 `T` 类型解释的那块内存”（由解引用的定义）。

思路：只要说明 `*p` 和 `x` 占用的是*同一段字节*，二者就是同一个变量。

详细推导：由 (2)(3)，`*p` 表示“从地址 A 开始的 `sizeof(T)` 个字节，按 `T` 解释”。由 (1) 和第一讲的地址范围关系，`x` 占用的也恰好是“从 A 开始的 `sizeof(T)` 个字节”，且类型同为 `T`。两者起点相同、长度相同、解释方式相同，所以是同一块内存。于是写 `*p = v` 就是把 v 写进 `x` 的那几个字节，即 `x` 的值变为 v。

同理，`*&x`：先由 `&x` 得到 A，再对 A 解引用得到“从 A 开始的 `T`”，就是 `x`；`&*p`：`*p` 是从 A 开始的对象，再取它的地址得到 A，即 `p` 的值。

一句话直观解释：`&` 是“问门牌号”，`*` 是“按门牌号上门”；先问门牌号再上门，找到的当然还是原来那户人家。 ■

2.3 同一个符号，三种身份

初学者最困惑的往往不是概念，而是 `*` 和 `&` 在不同位置意思不同：

写法	出现位置	含义
`int *p;`	声明语句中	`*` 是类型的一部分：“p 是指针”，**不是**解引用
`*p = 20;`	表达式中、放在指针前	解引用：“p 指向的那个变量”
`a * b`	两个数之间	乘法
`p = &a;`	表达式中、放在变量前	取地址
`int &r = a;`	声明语句中	引用（下一章内容，与指针不同，本章不展开）

表达式	类型	图 1.2 中的值	在图上对应什么
a	int	10	左边白框里的数
&a	int*	0x1000	左边白框上方的地址
p	int*	0x1000	右边蓝框里的数（箭头的起点）
*p	int	10	顺着箭头走到的白框（箭头的终点）
&p	int**	0x2000	右边蓝框上方的地址

例题 2.1　笔记 §7.2 示例代码的完整版

```
#include <iostream>
using namespace std;
int main()
{
    int a = 10;
    int *p;          // 1. 定义指针：数据类型 * 指针变量名
    p = &a;          //    让指针记录变量 a 的地址
    cout << "指针p为：" << p << endl;
    cout << "p指向的值：" << *p << endl;   // 2. 解引用：找到 p 指向的内存
    *p = 1000;       //    通过指针修改 a
    cout << "a = " << a << endl;
    return 0;
}
```

实际运行输出（g++，64 位 Linux；第一行的地址每次运行都可能不同）：

```
指针p为：0x7ffcc6ee782c
p指向的值：10
a = 1000
```

本题真正想说明：第 10 行没有出现 a，却改变了 a——这就是“间接访问”。对应图 2.1 的第 ④ 步。

易错点

1. **int* p, q;** 只定义了**一个**指针 p，q 是普通 int！因为 * 只属于紧跟它的那个名字。想要两个指针应写 **int** *p, *q;。
2. **类型要匹配**：**double** d; **int** *p = &d; 编译报错。int* 只能保存 int 变量的地址，因为解引用时要按 int 解释那块内存。
3. **定义了指针却不初始化**就解引用（图 2.1 第 ② 步的状态下写 *p = 5）——这是野

指针，后果不可预料。好习惯：定义时就初始化，如 `int *p = &a;` 或 `int *p = nullptr;`。

学完这里，你应该能够回答

1. `int *p = &a;` 中的 `*` 和 `*p = 5;` 中的 `*` 含义有何不同？
2. 设 `int a = 3; int *p = &a;`，写出 a、*p、*&a 的值，并说明哪些是“同一个变量”。
3. `*p = 1000` 之后，p 的值变了吗？为什么？
4. `int* p, q;` 中 q 是什么类型？

到这里，我们已经解决了什么？ 会定义指针、取地址、解引用了。**下一步还需要解决什么？** 指针本身也是变量，它占多大内存？`int*` 和 `double*` 大小一样吗？

第三讲　指针所占的内存空间：sizeof(p)

本讲回答两个问题： ① 一个指针变量占几个字节？② 为什么所有类型的指针大小都一样？

知识路线： 指针存的是地址 → 地址有多少位 → 指针就要多少字节 → 与指向的类型无关

3.1　为什么需要关心指针的大小

第八讲要用 `sizeof(arr) / sizeof(arr[0])` 求数组长度；而一旦数组被传进函数，`sizeof(arr)` 得到的却是*指针*的大小。要看懂这个“坑”，必须先知道指针有多大。

3.2　结论与推导

结论：指针的大小

同一个程序中，所有（普通数据）指针的大小相同，只取决于**目标平台的地址宽度**：

$$\texttt{sizeof(T*)} = \frac{\text{地址位数}}{8}\ \text{字节}$$

- 32 位程序：通常 4 字节
- 64 位程序：通常 8 字节

与 T 是 `char`、`int` 还是 `double` 无关。

推导：为什么 32 位地址需要 4 字节

已知：指针的值是一个地址；32 位平台上地址用 32 个二进制位表示。

第一步：32 个二进制位能表示 2^{32} 个不同的数，即可以区分 2^{32} 个字节，$2^{32}\ \text{B} = 4\times 2^{30}\ \text{B} = 4\ \text{GiB}$。这就是为什么 32 位程序最多直接使用约 4 GiB 内存。

第二步：要把任意一个 32 位地址存下来，需要 32 个二进制位 $= 32 \div 8 = 4$ 字节。

第三步：无论指向 char 还是 double，要存的都是“一个地址”，所以都是 4 字节。64 位平台同理：$64 \div 8 = 8$ 字节。
直观解释：门牌号的长度取决于“这座城市有多少户”，而不取决于“这户人家有多大”。■

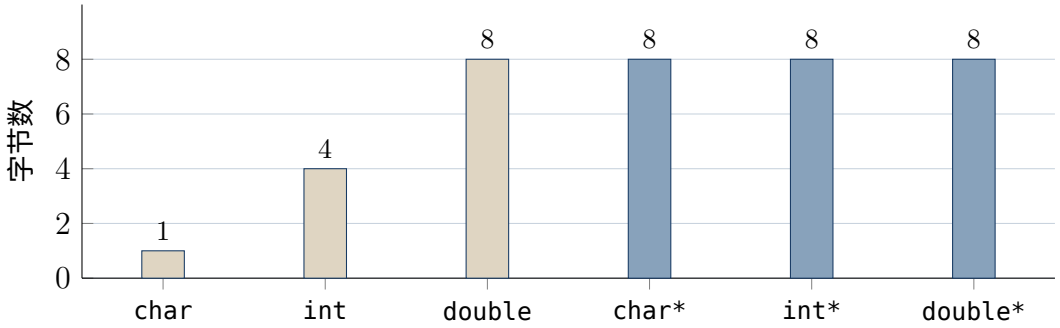


图 3.1　64 位 Linux 上 g++ 实测：变量大小各不相同（左三根），指针大小全部是 8（右三根）

图 3.1 的横轴是类型，纵轴是 **sizeof** 的结果。左边三根米色柱子高低不一——数据本身有大有小；右边三根蓝色柱子一样高——它们都只是“一个地址”。

例题 3.1　笔记 §7.3 示例

```
1 int a = 10;
2 int *p = &a;
3 int b = sizeof(p);              // 也可写 sizeof(int*)
4 cout << "int*字节数: " << b << endl;
5 cout << sizeof(*p) << endl;     // 注意: *p 是 int, 结果是 4
```

64 位程序输出 8 和 4。第 5 行是补充的对比：**sizeof**(p) 问的是“门牌号多大”，**sizeof**(*p) 问的是“这户人家多大”。

易错点

1. 把 **sizeof**(p) 与 **sizeof**(*p) 混为一谈：前者是指针大小，后者是所指对象的大小。
2. 认为“64 位操作系统上指针一定是 8 字节”：取决于**程序**按哪个平台编译（见原图纠错 ⑥）。

拓展联想

以下内容只用于建立知识联系，当前阶段不要求掌握。

为什么 long 在 64 位 Linux 上是 8 字节、在 64 位 Windows 上是 4 字节，而两者的指针都是 8 字节？这与“数据模型”（LP64 与 LLP64）有关，以后学习计算机组成原理和系统编程时会再遇到。

到这里，我们已经解决了什么？ 知道了指针本身的大小。**下一步还需要解决什么？** 指针里存的地址如果不合法会怎样？指针暂时没有东西可指时，应该存什么？

第四讲　空指针与野指针：哪些地址不能碰

本讲回答三个问题： ① 指针暂时无处可指时应该存什么？② 什么样的指针是“野”的？③ 为什么二者都不能解引用？

知识路线： 未初始化的危险 → 空指针（安全的“无指向”）→ 野指针（危险的“乱指向”）→ 统一原则：只访问自己申请的空间

4.1　为什么需要空指针

第二讲图 2.1 第 ② 步已经暴露了问题：`int *p;` 定义之后，p 里是一个*不确定*的值，看起来像地址，却不知道指向哪里。我们需要一个**约定好的特殊值**，表示“此刻不指向任何东西”，并且程序可以检查它。这个值就是空指针。

定义：空指针

值为**空指针常量**的指针称为空指针，它**不指向任何对象**。在 C++ 中可以写成：

- `int *p = nullptr;`　（C++11 起，推荐）
- `int *p = NULL;`　（笔记写法，C 风格）
- `int *p = 0;`

用途：初始化指针变量；表示“没有指向”，并可用 `if (p != nullptr)` 检查。
注意：空指针不可解引用——`*p` 的读和写都是非法的。

> **教学补全：空指针的“编号 0”和 nullptr**
>
> 原始笔记未完整展示该部分，以下按照本课程常见教材与 C++ 标准表述补充。
>
> 笔记定义为“指针变量指向内存中编号为 0 的空间”。这是一种便于理解的说法：在常见平台上，空指针的值确实就是地址 0，而操作系统通常不会把地址 0 附近的内存分配给程序，所以访问它会被拦截。更准确地说，C++ 标准规定的是“空指针不指向任何对象，解引用它是**未定义行为**”，至于它在机器里用什么数表示由实现决定。
> `NULL` 是一个宏，通常被定义为 `0`；而 `nullptr` 是 C++11 引入的**专用**关键字，只能当指针用，不会被误当成整数 0。新代码建议使用 `nullptr`。

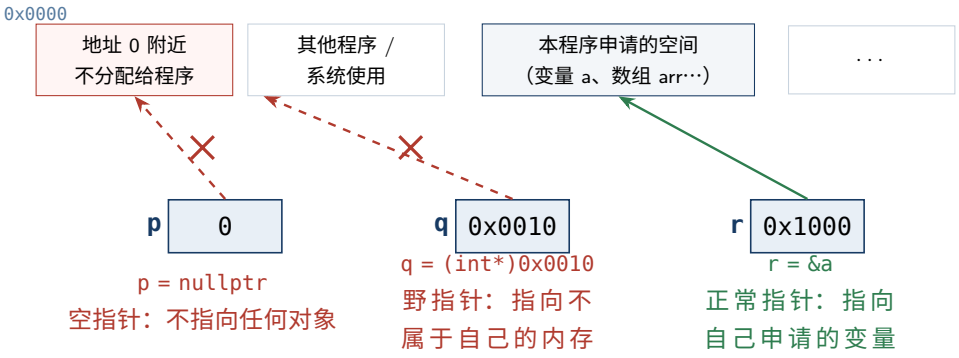


图 4.1　三种指针在“内存地图”上的位置：只有指向本程序申请空间的指针才能解引用

图 4.1 上方是一条简化的“内存地图”：最左边红色区域是地址 0 附近，通常不分配给程序；蓝色区域是本程序通过定义变量申请到的空间。下方三个指针中，只有 r 的箭头（绿色实线）落在蓝色区域内。

定义：野指针

指向**非法内存空间**（不属于本程序可以合法访问的对象）的指针称为野指针。常见来源：

(1) **未初始化**：`int *p;`（局部变量，值不确定）；

(2) **随意写一个地址**：`int *p = (int*)0x0010;`（笔记示例，`(int*)` 把整数强制转换成指针）；

(3) **越界**：指针移到数组范围之外（第六讲、原图纠错 ①）；

(4) **失效**（拓展）：指向的变量已经不存在（例如函数返回后其局部变量已被销毁）。

为什么“不要访问”：未定义行为

笔记的红字总结非常关键：**“空指针和野指针都不是我们申请的空间，因此不要访问。”** C++ 标准把这类访问称为**未定义行为**（undefined behavior）：标准不规定结果。实际表现可能是立即崩溃（段错误）、输出一个莫名其妙的数、悄悄改坏别的数据，甚至“看起来一切正常”。最后一种最危险——程序今天没出错，不代表它是对的。

例题 4.1　笔记 §7.4 两段示例及其运行结果

```
int *p = NULL;        // 空指针用于给指针变量初始化
// *p = 100;          // 空指针不可访问：Linux 上通常 → Segmentation fault

int *q = (int *)0x0010;      // 野指针：0x0010 不是我们申请的空间
// cout << *q << endl;       // 同样不要访问
```

安全的写法：先判断再使用。

```
int a = 5;
int *p = nullptr;              // 暂时无处可指
p = &a;                        // 有了合法对象再指向它
if (p != nullptr) cout << *p; // 使用前检查
```

本题真正想说明：空指针的价值不在于“能访问”，而在于“**可以被检查**”。野指针的值看起来和正常地址没有区别，程序无法检查出来，所以更危险。

比较维度	空指针	野指针
值	确定的特殊值（nullptr/NULL/0）	不确定或任意的地址
是否指向对象	明确不指向任何对象	“看起来”指向某处，但那里不属于你
能否检查出来	能：if (p == nullptr)	不能：程序无法判断一个地址是否“野”
能否解引用	不能（未定义行为）	不能（未定义行为）
怎么产生	程序员**有意**赋值	通常是**疏忽**：未初始化、越界、乱写地址
地位	一种良好习惯	一种错误

易错点

1. 以为“空指针访问一定会报段错误”，于是把崩溃当作检查手段。正确做法是访问前主动判断。
2. 以为“程序没崩溃就说明没有越界”。越界读写可能恰好读到别的变量，程序照常运行但结果是错的。
3. 把“空指针”和“指向值为 0 的变量”混淆：int z = 0; int *p = &z; 中 p 不是空指针，*p 完全合法。

学完这里，你应该能够回答

1. 空指针和野指针共同点是什么？最本质的区别是什么？
2. 列出野指针的三种常见来源。
3. int z = 0; int *p = &z;，p 是空指针吗？
4. 为什么推荐 nullptr 而不是 NULL？

到这里，我们已经解决了什么？ 知道了指针可以“指向谁”、什么情况下“不能碰”。**下一步还需要解决什么？**指针既能改“指向”，又能通过解引用改“值”，这种力量有时太大了。

能不能*限制*指针只做其中一件事？这就是 const 修饰指针。

第五讲　const 修饰指针：给指针“上锁”

本讲回答三个问题： ① 一个指针能改变哪两样东西？② const 放在不同位置分别锁住哪一样？③ 怎样一眼读出一个声明锁住了什么？

知识路线：指针的两种“权力”→ 锁值（常量指针）→ 锁指向（指针常量）→ 两个都锁 → 读法口诀

5.1　为什么需要 const 修饰指针

回顾图 2.1：指针有两种“权力”——

(i) **改指向**：`p = &b;`，让箭头指向别处（改的是 p 自己）；
(ii) **改值**：`*p = 20;`，顺着箭头修改对方（改的是 *p）。

第八讲的 PrintArray 函数只需要*读*数组，如果不小心写了 `arr[i] = 0;`，原数组就被破坏了。我们希望让编译器帮忙把关：**声明时就说清楚这个指针可以做什么、不可以做什么**，违反时直接编译报错。这就是 const 修饰指针的用途。

5.2　先看直观图像：锁在哪里

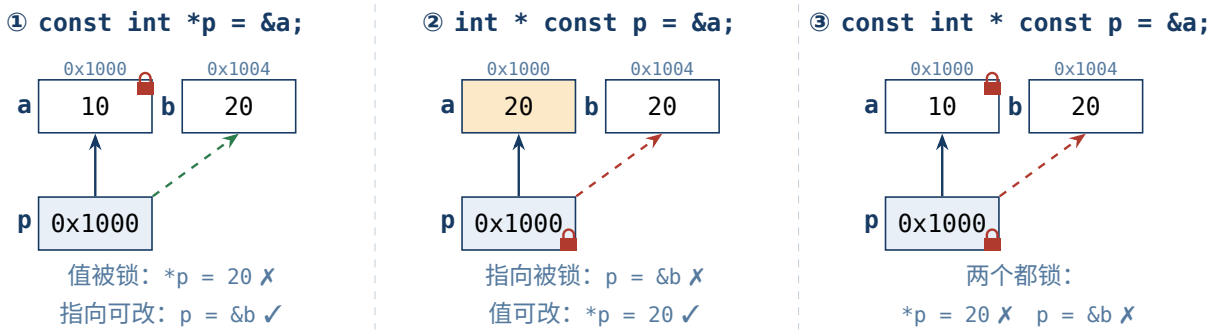


图 5.1　笔记 §7.5 图示的重绘（已改正 b 的值为 20）：红锁在“值”上，还是在“指针”上

读图：每个小图中上排是变量 a、b，下排蓝框是指针 p。**红锁画在目标方框上**表示“不能通过 p 改值”；**红锁画在指针方框上**表示“p 自己不能改”，所以指向 b 的箭头是红色虚线（不允许）。图 ② 中 a 已被 `*p = 20` 改成 20（黄色）。

5.3 正式定义

定义：const 修饰指针的三种情况（笔记 §7.5）

声明	笔记中的名称	常见英文说法	改指向 p=&b	改值 *p=20
`const int *p = &a;`	常量指针	pointer to const	✓	✗
`int * const p = &a;`	指针常量	const pointer	✗	✓
`const int * const p = &a;`	（两者都修饰）	const pointer to const	✗	✗

核心关系：读法口诀：看 const 在 * 的哪一边

`const` 在 * 左边 ⇒ 锁“值”　　`const` 在 * 右边 ⇒ 锁“指向”

等价的“从右往左读”法：`int * const p` 读作“p 是 const 的、指向 int 的指针”；`const int *p` 读作“p 是指针，指向 const int”。`int const *p` 与 `const int *p` 完全相同（const 仍在 * 左边）。

名称容易混：以“位置”为准，不要死记名字

“常量指针”“指针常量”这两个中文名称，不同教材的叫法**并不统一**，甚至恰好相反。本讲义沿用笔记的叫法，但考试和写代码时，请一律用上面的“左锁值、右锁指向”判断，不要只凭名称作答。

5.4 为什么是这样：从类型推导

推导：为什么 `const int *p` 不能 `*p = 20` 却能 `p = &b`

已知：C++ 规定，被声明为 `const` 的对象不能出现在赋值号左边，否则编译错误。

第一步（看 `*p`）：声明 `const int *p` 的含义是“`*p` 的类型为 `const int`”。所以 `*p = 20` 是给一个 `const int` 赋值，被编译器拒绝。g++ 报错：`assignment of read-only location '* p1'`。

第二步（看 `p`）：`p` 本身的类型是 `const int*`，这个类型*最外层*没有 const，`p` 是普通变量，所以 `p = &b` 合法。

第三步（对称情况）：`int * const p` 中 const 修饰的是 `p` 本身，`p = &b` 报错 `assignment of read-only variable 'p'`；而 `*p` 的类型是普通 `int`，可以赋值。

结论：const 写在 * 左边，修饰的是“*p”（指向的东西）；写在 * 右边，修饰的是“p”（指针自己）。上面的报错信息均为 g++ 实测。 ■

教学补全：两个常被忽略的边界情况

原始笔记未完整展示该部分，以下按照本课程常见教材与 C++ 标准表述补充。

(1) 常量指针不会让原变量变成常量。

```
int a = 10;
const int *p = &a;
// *p = 30;   // 错误：不能“通过 p”修改
a = 30;       // 正确：a 本身不是 const，直接改完全可以
```

所以 `const int *p` 的准确含义是“**不能通过 p 修改所指对象**”，而不是“所指对象永远不变”。

(2) 去掉条件会怎样：常量的地址只能交给“常量指针”。

```
const int c = 5;
// int *q = &c;        // 错误：若允许，就能用 *q = 6 修改常量 c
const int *q = &c;     // 正确
```

这说明 `const int*` 中的 `const` 不是可有可无的装饰：它正是编译器用来保护常量的“通行证”。

例题 5.1　笔记 §7.5 示例代码（已改正注释）

```
int a = 10, b = 20;
// const 修饰常量（指针常量）：指向不可改，值可改
int * const p = &a;
*p = 20;                    // 合法
// p = &b;                  // 不合法
// const 修饰指针（常量指针）：指向可改，值不可改
const int *p1 = &b;
p1 = &a;                    // 合法
// *p1 = a;                 // 不合法
// const 既修饰指针又修饰常量
const int * const p2 = &a;
// p2 = &b 与 *p2 = b 均非法    （笔记原写 p1，已改正）
```

本题真正想说明：判断一行是否合法，只需两步——① 这一行改的是 p 还是 *p？② 对应的那一侧有没有 const？

判断步骤（三步法）

第一步找到 *；**第二步**看 const 在 * 左边、右边还是两边都有；**第三步**看被检查的语句改的是 *p（值）还是 p（指向），对应一侧有锁就不合法。

学完这里，你应该能够回答

1. `int const *p` 锁的是什么?
2. `const int *p = &a;` 之后，`a = 5;` 合法吗? `*p = 5;` 呢?
3. 为什么 `const int c = 5; int *q = &c;` 会编译失败?
4. 如果要写一个“只读取数组、不修改数组”的函数，形参应该怎样写?

到这里，我们已经解决了什么? 会用 const 控制指针的权力了。**下一步还需要解决什么?** 目前指针都只指向单个变量。数组是一排连续的变量——指针能不能“一格一格地走”过去?

第六讲　指针和数组：让指针在数组上移动

本讲回答三个问题： ① 为什么 `int *p = arr;` 不用写 `&`? ② `p++` 究竟让地址增加了多少? ③ 用指针遍历数组时，边界在哪里?

知识路线： 数组连续存放 $\rightarrow$ 数组名转换为首元素指针 $\rightarrow$ 指针算术 $p+k$ $\rightarrow$ `arr[i]` $\equiv$ `*(arr+i)` $\rightarrow$ 遍历与越界

6.1　为什么需要用指针访问数组

数组元素在内存中**紧挨着**存放。既然知道第一个元素在哪里、每个元素多大，就能算出任何一个元素的位置。指针正是做这种“按位置访问”的工具——第八讲把数组交给函数时，函数拿到的恰恰只有“第一个元素在哪里”。

6.2　数组名与首元素地址

定义：数组到指针的转换

在大多数表达式中，数组名 `arr` 会自动转换为**指向首元素的指针**，即

$$\texttt{arr} \longrightarrow \texttt{\&arr[0]} \quad (\text{类型为} \texttt{int*})$$

因此 `int *p = arr;` 与 `int *p = &arr[0];` 完全等价。
例外（本章会遇到的）：`sizeof(arr)` 中不转换，得到整个数组的字节数。另外数组名不能被赋值，`arr++` 是错误的。

6.3 核心关系：指针加 1 到底加了多少

核心关系：指针算术

设 `T *p` 指向某数组中的一个元素，k 为整数，则 `p + k` 指向其后第 k 个元素，其地址为

$$\boxed{\text{地址}(\texttt{p}+k) \;=\; \text{地址}(\texttt{p}) \;+\; k \times \texttt{sizeof(T)}}$$

特别地，`p++` 使地址增加 `sizeof(T)` 字节；对 `int*`（`int` 为 4 字节）即“向后偏移 4 个字节”（笔记红字）。

使用条件：`p+k` 必须仍位于同一数组内（或恰好指向最后一个元素的下一个位置，但那个位置不能解引用）。超出范围就是越界。

推导：步长为什么是 `sizeof(T)`，以及 `arr[i]` ≡ `*(arr+i)`

已知：数组 `T arr[n]` 的 n 个元素连续存放，每个占 `sizeof(T)` 字节，`arr[0]` 的地址为 A。

第一步：`arr[0]` 占 $[A,\ A+s-1]$（记 $s=\texttt{sizeof(T)}$），紧接着的 `arr[1]` 从 $A+s$ 开始；以此类推，`arr[i]` 的地址为 $A+i\cdot s$（数学归纳：每多一个元素，起点后移 s）。

第二步：设计者希望“`p+1` 指向下一个元素”，由第一步，下一个元素比当前元素的地址大 s，所以指针加 k 必须让地址加 $k\cdot s$，而不是加 k 字节。这就是指针必须带类型的原因——**类型决定步长**。

第三步：由数组到指针的转换，`arr` 等于 `&arr[0]`，地址为 A；于是 `arr + i` 的地址为 $A+i\cdot s$，正是 `arr[i]` 的地址。由第二讲，对它解引用就得到 `arr[i]` 本身：

$$\texttt{*(arr + i)} \;\equiv\; \texttt{arr[i]}.$$

事实上，C++ 标准正是把下标运算 `E1[E2]` 定义为 `*((E1)+(E2))`。所以对指针也可以写下标：`p[i]` 就是 `*(p+i)`。 ■

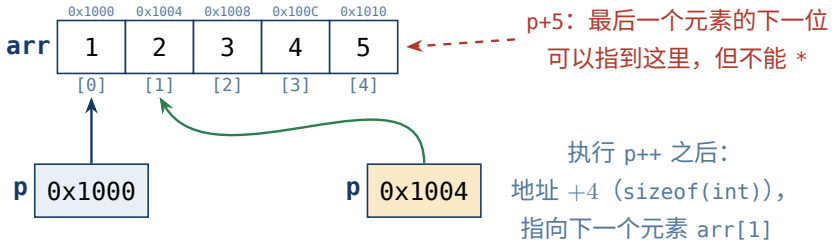


图 6.1　p++ 不是“地址加 1”，而是“走到下一个元素”——`int*` 每步 4 字节

图 6.1 中数组 5 个元素首尾相接，上方是示意地址，相邻两个元素恰好差 `0x04`。初始时 `p` 指向 `arr[0]`（深蓝箭头）；执行 `p++` 后，`p` 的值由 `0x1000` 变为 `0x1004`，指向 `arr[1]`（绿箭头）。

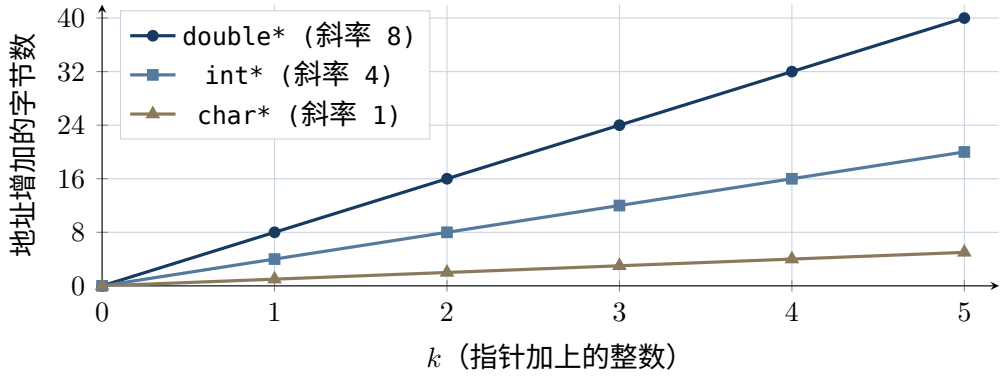


图 6.2　$p+k$ 的地址增量与 k 成正比，斜率就是 sizeof(T)

图 6.2 的横轴是加到指针上的整数 k，纵轴是地址实际增加的字节数。三条直线都过原点，斜率分别是 8、4、1——这就是公式 $k \times$ sizeof(T) 的图像：**同样是 $p+3$，不同类型的指针走过的字节数不同**。

例题 6.1　笔记 §7.6 示例：访问第一、第二个元素

```cpp
int arr[] = {1,2,3,4,5,6,7,8,9,10};
int *p = arr;                          // 数组名就是首元素地址
cout << "第一个元素：" << *p << endl;
cout << p << endl;
p++;                                   // 向后偏移 4 个字节（sizeof(int)）
cout << p << endl;
cout << "第二个元素为：" << *p << endl;
```

实测输出：第一个元素：1，0x7ffd86321fa0，0x7ffd86321fa4，第二个元素为：2。两个地址的末位由 a0 变为 a4，恰好相差 4，验证了公式。

例题 6.2　通过指针遍历数组（改正越界版本）

```cpp
int arr[] = {1,2,3,4,5};
int len = sizeof(arr) / sizeof(arr[0]);   // 20 / 4 = 5
int *p = arr;
for (int i = 0; i < len; i++)             // 笔记原写 i < 10，越界
{
    cout << *p << endl;
    p++;
}
```



本题真正想说明：指针遍历时编译器**不会**检查是否越界，边界完全由程序员负责。用 `sizeof(arr)/sizeof(arr[0])` 计算长度，是最简单的防护。

易错点

1. **认为 `p++` 是地址加 1。**对 `int*` 是加 4 字节（一个元素），见图 6.2。
2. **混淆 `p++` 与 `(*p)++`。**前者移动指针，后者把当前元素的值加 1。
3. **对数组名做自增。**`arr++` 编译错误；要移动请先 `int *p = arr;` 再移动 `p`。
4. **忘了越界没有报错。**越界读写可能不崩溃，但结果不可信（原图纠错 ①）。

拓展联想

以下内容只用于建立知识联系，当前阶段不要求掌握。

`*p++` 表示“先取 `*p`，再让 `p` 后移”（后置 `++` 优先级更高，但返回旧值）。这种紧凑写法在 C 语言库代码中很常见，初学阶段建议分成两行写。

学完这里，你应该能够回答

1. `int arr[5]` 首地址为 `0x2000`，`arr + 3` 的地址是多少？
2. `*(arr+2)` 和 `arr[2]` 是什么关系？`*arr + 2` 呢？
3. 若 `double *q` 指向 `0x3000`，`q + 2` 的地址是多少？
4. 为什么说“类型决定步长”？

到这里，我们已经解决了什么？ 指针可以在数组上移动、访问任意元素。**下一步还需要解决什么？**指针最重要的用途之一是和函数配合：怎样让一个函数修改调用者的变量？

第七讲　指针和函数：值传递与地址传递

本讲回答三个问题： ① 为什么 `swap(a, b)` 交换不了 `main` 中的 `a`、`b`？② 传地址为什么就能改？③ 什么时候该传地址？

知识路线：回顾值传递（形参是拷贝）→ 证明值传递改不了实参 → 地址传递（拷贝的是地址）→ 通过解引用改实参 → 选择原则

7.1 回顾：值传递

例题 7.1　笔记 §7.7 值传递示例

```cpp
void swap(int a, int b)        // 形参 a、b
{
    int temp = b;
    b = a;
    a = temp;
}
int main()
{
    int a = 10;
    int b = 20;
    swap(a, b);                // 实参 a、b
    cout << "a=" << a << " b=" << b << endl;   // 输出 a=10 b=20
    return 0;
}
```

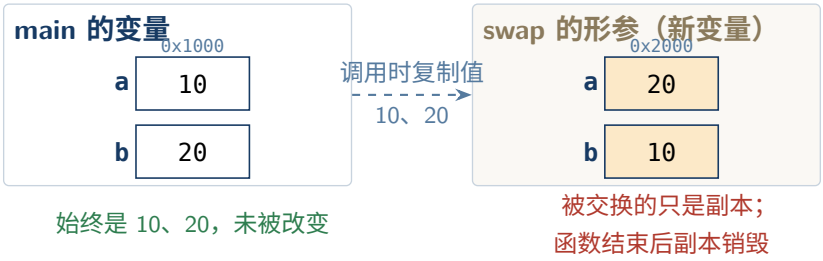


图 7.1　值传递：swap 中的 a、b 是另外两个变量（地址不同），交换它们与 main 无关

结论：值传递不能修改实参

若函数形参是普通变量（非指针、非引用），则在函数体内对形参的任何赋值，都**不会**改变调用处的实参。

证明 / 推导

已知：C++ 在调用 `swap(a, b)` 时，为形参创建**新的**变量，并用实参的*值*初始化它们（这就是“值传递”的定义）。

第一步：形参 `a` 与 `main` 中的 `a` 是两个不同的对象，占用不同的内存（图 7.1 中 `0x1000` 与 `0x2000`）。

第二步：赋值语句只修改赋值号左边那个对象所占的字节。`swap` 函数体中所有赋值的左边都是形参 `a`、`b` 或局部变量 `temp`，它们都在 `swap` 自己的内存中。

第三步：因此 `main` 中 `a`、`b` 所占的字节从未被写过，值保持 10、20。函数返回后形参被销毁，交换结果随之消失。

> **直观解释**：你给朋友一份文件的*复印件*，他在复印件上怎么改，你手里的原件都不会变。 ■

7.2 地址传递：把“原件的位置”交出去

例题 7.2　笔记 §7.7 地址传递示例

```
void swap_p(int *p1, int *p2)
{
    int temp = *p1;           // 取出 a 的值
    *p1 = *p2;                // 把 b 的值写进 a
    *p2 = temp;               // 把原来 a 的值写进 b
}
int main()
{
    int a = 10;
    int b = 20;
    swap_p(&a, &b);           // 传入 a、b 的地址
    cout << "a=" << a << " b=" << b << endl;   // 输出 a=20 b=10
    return 0;
}
```

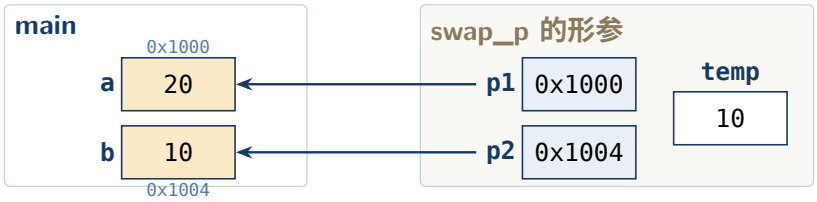


图 7.2　地址传递：形参 p1、p2 保存的是 main 中 a、b 的地址，*p1、*p2 就是 main 的 a、b

步骤	执行的语句	temp	a（即 *p1）	b（即 *p2）
0	进入函数	—	10	20
1	`int temp = *p1;`	10	10	20
2	`*p1 = *p2;`	10	**20**	20
3	`*p2 = temp;`	10	20	**10**

关键认识：“地址传递”其实也是值传递

仔细看图 7.2：p1、p2 也是 swap_p 新建的变量，它们同样是*拷贝*——只不过拷贝的值是**地址**。拷贝来的地址和原地址相同，所以顺着它解引用，找到的是 main 中的原变量。由第二讲的 *p ≡ x，*p1 = *p2 就是在修改 main 的 a。

> 推论（常考）：如果在函数中修改的是 *指针本身*（如 `p1 = nullptr;`），调用者手里的指针**不受影响**——因为那只是改了副本。改“指向的东西”才会影响外部。

比较维度	值传递	地址传递
形参类型	`int a`	`int *p`
调用写法	`swap(a, b)`	`swap_p(&a, &b)`
复制了什么	实参的值（10、20）	实参的地址（0x1000、0x1004）
函数内如何修改	改形参自己	改 `*p`，即实参本身
能否改变实参	不能	能
适用场景	只需要读实参的值	需要修改实参；或数据很大、不想复制（配合 const）

易错点

1. 调用时漏写 `&`：`swap_p(a, b)` 把 `int` 传给 `int*` 形参，编译报错。
2. 函数内写成 `int temp = p1; p1 = p2; p2 = temp;`（交换的是两个指针副本），实参不变。必须写 `*`。
3. 以为“传了指针就一定能改外部”：只有改 `*p` 才影响外部；改 `p` 本身不影响（见 B 组第 27 题）。

拓展联想

以下内容只用于建立知识联系，当前阶段不要求掌握。

下一章将学习“引用”：`void swap_r(int &a, int &b)` 也能交换实参，而调用时不需要写 `&`。引用在底层通常也是用地址实现的，可以看作“更安全、写法更简洁的指针”。

学完这里，你应该能够回答

1. 用自己的话证明：值传递的函数为什么改不了实参？
2. `swap_p` 中 `temp` 为什么必须是 `int` 而不是 `int*`？
3. “地址传递也是值传递”这句话对吗？怎么理解？
4. 函数内执行 `p1 = nullptr;`，调用者的指针会变成空指针吗？

到这里，我们已经解决了什么？ 函数可以通过指针修改外部数据了。**下一步还需要解决什么？** 把前面所有内容合在一起：写一个函数，接收一个数组，把它原地排好序——这就是笔记 §7.8 的综合案例。

第八讲　指针、数组和函数：封装冒泡排序

本讲回答四个问题： ① 数组怎样传给函数？函数里还能用 `sizeof` 求长度吗？② 冒泡排序每一轮在做什么？③ 两层循环的边界 `n-1` 和 `n-i-1` 从哪里来？④ 为什么冒泡排序一定能排好序？

知识路线：数组传参 = 传首地址 → 必须另传长度 → 相邻比较交换 → 推导循环边界 → 正确性证明 → 比较次数

8.1　任务

笔记案例：**封装一个函数，利用冒泡排序，实现对整型数组的升序排序**。例如数组 `{3, 7, 9, 12, 5, 1}` 排序后为 `1 3 5 7 9 12`。

8.2　第一步：数组怎样传进函数

定义：数组作函数参数

调用 `BubbleSort(arr, len)` 时，实参 `arr` 按第六讲的规则转换为 `&arr[0]`，所以形参只能收到一个**地址**。以下两种形参写法完全等价：

```
void BubbleSort(int *arr, int n)  ⟺  void BubbleSort(int arr[], int n)
```

函数通过这个地址直接操作原数组（这是第七讲“地址传递”），所以排序结果会保留在 `main` 的数组中。

结论：数组长度必须由调用者计算并传入

在定义数组的地方，$\texttt{len} = \dfrac{\texttt{sizeof(arr)}}{\texttt{sizeof(arr[0])}}$ 等于元素个数；但在函数内部，`sizeof(arr)` 只是**指针的大小**（64 位程序为 8），不能再这样求长度。

推导

在 main 中：`int arr[] = {3,7,9,12,5,1};` 有 6 个 `int`。`sizeof(arr)` 属于第六讲所说的“不转换”的例外，得到整个数组的字节数 $6 \times 4 = 24$；`sizeof(arr[0])` $= 4$，于是 $24 \div 4 = 6$。一般地，$\dfrac{n \cdot \texttt{sizeof(int)}}{\texttt{sizeof(int)}} = n$。

在函数中：形参 `arr` 的类型是 `int*`（即使写成 `int arr[]` 也是如此），由第三讲，`sizeof(arr)` $= 8$，$8 \div 4 = 2$，与真实长度无关。若形参写成 `int arr[]`，g++ 还会给出警告 `sizeof on array function parameter ... will return size of int*`。所以长度 `n` 必须作为第二个参数传进来。 ■

8.3 第二步：冒泡排序在做什么

定义：冒泡排序（升序）

从头到尾依次比较**相邻**两个元素 `arr[j]` 与 `arr[j+1]`，若前者大于后者就交换。这样“走一遍”称为**一轮**，一轮结束时，参与比较的元素中最大的那个会被换到最后，就像气泡上浮。对剩下的元素重复进行，直到全部有序。



图 8.1　对 {5,3,7,8,2} 做第 1 轮（$i = 0$）：4 次相邻比较，把最大值 8“冒”到最后

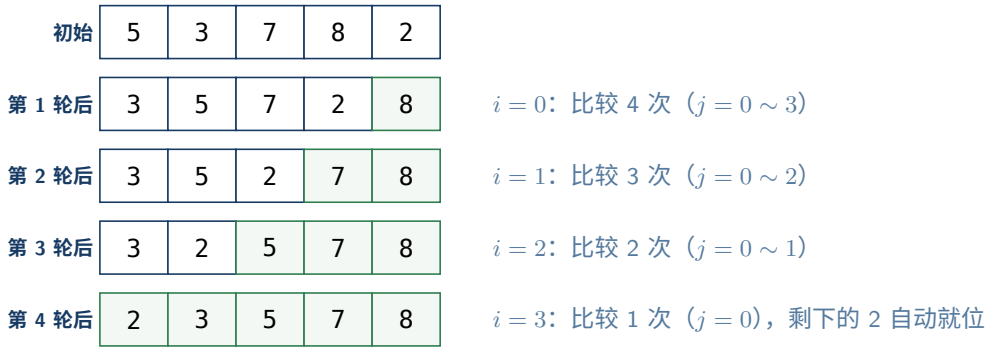


图 8.2　每轮结束时的状态：绿色部分（已就位）每轮向左增长一格

图 8.1 展示一轮内部的每一次比较；图 8.2 只看每一轮结束时的数组。注意绿色区域：第 1 轮后末尾 1 个就位，第 2 轮后末尾 2 个就位……这正是下面推导循环边界的依据。

8.4 第三步：推导两层循环的边界

核心关系：冒泡排序的循环边界（n 个元素）

```
外层:  for (int i = 0; i < n - 1; i++)
内层:  for (int j = 0; j < n - i - 1; j++)
```

i：已经完成的轮数（从 0 开始），也等于末尾已就位的元素个数；j：本轮正在比较的一对中左边元素的下标。

推导（笔记 §7.8 “理解”部分的补全）

外层为什么是 $n-1$ 轮：每轮至少让一个元素就位（下面的定理保证）。$n-1$ 轮后，末尾 $n-1$ 个元素已就位，剩下的唯一一个元素必然是最小值，位于 `arr[0]`，也已就位。所以最多需要 $n-1$ 轮，i 取 $0,1,\ldots,n-2$，即 `i < n-1`。

内层为什么是 $j < n-i-1$：

1. 第 i 轮开始时，末尾 i 个元素已就位，需要处理的是下标 $0 \sim n-1-i$ 的部分；
2. 比较的是 `arr[j]` 与 `arr[j+1]`，右边元素下标 $j+1$ 最大为 $n-1-i$，所以 $j \le n-2-i$；
3. $j \le n-2-i$ 等价于 $j < n-i-1$。

代入笔记的例子：$n=5$，$i=0$ 时最后比较的是 `arr[3]` 与 `arr[4]`，$j_{\max}=3$，即 $j<4=n-1$；每进行一轮，需要比较的少一个，所以一般地 $j<n-i-1$。

比较次数：第 i 轮比较 $n-1-i$ 次，总次数

$$\sum_{i=0}^{n-2}(n-1-i) = (n-1)+(n-2)+\cdots+1 = \frac{n(n-1)}{2}.$$

例如 $n=5$ 时为 10 次（图 8.2：$4+3+2+1$），$n=6$ 时为 15 次。 ■

结论：冒泡排序的正确性

对任意 $n \ge 1$ 个整数，按上述边界执行冒泡排序后，数组按升序排列。

证明 / 推导

要证明：对 $i=0,1,\ldots,n-2$，第 i 轮结束后，下标 $n-1-i \sim n-1$ 的 $i+1$ 个位置上是整个数组中最大的 $i+1$ 个数，并已按升序排列（称为“性质 $P(i)$”）。

引理（一轮内部）：在某一轮中，比较完 `arr[j]` 与 `arr[j+1]` 后，`arr[j+1]` 等于本轮参与部分 `arr[0..j+1]` 中的最大值。

证明：对 j 归纳。$j=0$：比较并在需要时交换后，`arr[1]` 是 `arr[0]`、`arr[1]` 中较大的。设比较完第 $j-1$ 对后 `arr[j]` 是 `arr[0..j]` 的最大值；比较第 j 对后，`arr[j+1]` 变为 $\max($`arr[j]`$,$`arr[j+1]`$)$，也就是 `arr[0..j+1]` 的最大值。引理得证。

主证明（对轮数归纳）：第 i 轮的内层循环最后一次 $j=n-i-2$，由引理，结束时 `arr[n-1-i]` 是 `arr[0..n-1-i]` 的最大值。当 $i=0$ 时，它就是全体最大值，$P(0)$ 成立。若 $P(i-1)$ 成立，则 `arr[0..n-1-i]` 恰好是除去最大 i 个数后剩下的数，其中最大者即全体第 $i+1$ 大的数，被放到 `arr[n-1-i]`，且它不大于后面已就位的数，所以 $P(i)$ 成立。

结论：$P(n-2)$ 表明下标 $1 \sim n-1$ 是最大的 $n-1$ 个数且有序，剩下的 `arr[0]` 是最小值，于是整个数组有序。

一句话直观解释：每一轮都像“擂台赛”，最大的一路赢到最后；赢过的就不再参赛。■

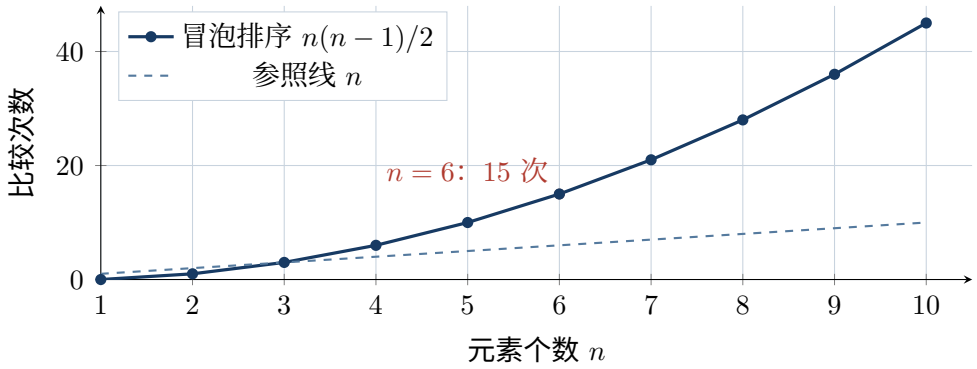


图 8.3　比较次数随 n 按二次函数增长：n 翻一倍，比较次数约变为 4 倍

图 8.3 横轴为元素个数，纵轴为比较次数。实线（冒泡排序）是抛物线，虚线是直线 $y=n$ 作参照：n 较大时实线远高于虚线。这说明冒泡排序适合教学和小规模数据，大规模数据需要更快的算法（后续“数据结构”课程）。

8.5　第四步：完整程序（已改正笔记中的两处错误）

```cpp
#include <iostream>
using namespace std;

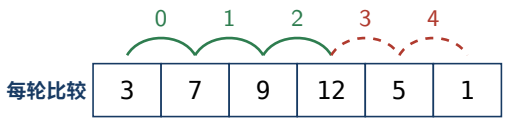
// 冒泡排序：arr 为数组首地址，n 为元素个数
void BubbleSort(int *arr, int n)
{
    for (int i = 0; i < n - 1; i++)            // 共 n-1 轮
    {
        for (int j = 0; j < n - i - 1; j++)    // 笔记原写 n-j-1，错误
        {
            if (arr[j] > arr[j + 1])           // 笔记原缺少括号
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// 打印数组：只读，用 const 保护（第五讲）
void PrintArray(const int *arr, int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
```

```
    cout << endl;
}

int main()
{
    int arr[] = {3, 7, 9, 12, 5, 1};
    int len = sizeof(arr) / sizeof(arr[0]);    // 24 / 4 = 6
    BubbleSort(arr, len);                      // 传入首地址和长度
    PrintArray(arr, len);                      // 输出：1 3 5 7 9 12
    return 0;
}
```

这段代码一次用上了本章全部主线：**数组名即首地址**（第六讲）→ **地址传递使函数能修改原数组**（第七讲）→ **sizeof 求长度必须在 main 中进行**（第三讲）→ **const 保护只读参数**（第五讲）。与笔记相比，PrintArray 的形参加了 const，输出改为同一行，属于教学补全。



错误版本 j < n-j-1（$n=6$）：
$j < 5-j \iff j \le 2$，只比较绿色的 3 对，
红色虚线的 12,5、5,1 永远不比较
⇒ 实测输出仍为 3 7 9 12 5 1

图 8.4　笔记封装版本的错误：内层条件写成 n-j-1，每一轮都只比较前 3 对

易错点

1. **内层边界写错**：j < n-j-1（笔记原误）只比较前半部分；j < n-1-i 写成 j < n-i 则会让 arr[j+1] 越界。
2. **在函数内用 sizeof(arr) 求长度**：得到的是指针大小，见上面的推导。
3. **交换时顺序写错**：必须借助 temp，写成 arr[j] = arr[j+1]; arr[j+1] = arr[j]; 会丢失一个值。
4. **升序/降序符号弄反**：升序用 > 时交换；降序改成 <。

拓展联想

以下内容只用于建立知识联系，当前阶段不要求掌握。

实际开发中可直接使用标准库 std::sort(arr, arr + len);——注意它的参数也是两个指针：“首元素地址”和“最后一个元素的下一个位置”，正是第六讲图 6.1 中的 p+5。这种“左闭右开”的表示法会贯穿整个 C++ 标准库。

学完这里，你应该能够回答

1. 为什么 BubbleSort 能修改 main 中的数组，而第七讲的 swap(a,b) 却不能修改实参？

2. 对 8 个元素做冒泡排序，外层几轮？总共比较几次？
3. 写出 `j < n-i-1` 的推导过程。
4. 为什么函数中的 `sizeof(arr)` 不能用来求数组长度？

到这里，我们已经解决了什么？ 从“地址是什么”出发，一路走到了“用指针把数组交给函数去排序”，第七章的主线全部打通。接下来用一张知识地图回顾全章，然后进入分层练习。

本章知识地图与总自检



图 9.1　第七章知识地图：每个分支的叶子是该讲“必须记住的一句话”

全章学习自检（不计入正式练习）

1. 为什么需要指针？说出至少两个“只用变量名做不到”的场景。
2. &、* 分别是什么运算？为什么 `*p` 就是 `a` 本身？
3. 为什么 `int*` 与 `double*` 大小相同？为什么 `p+1` 走过的字节数却不同？

4. 空指针和野指针有什么共同点？哪一个可以被程序检查出来？
5. 看到 `const int * const p`，你能在 3 秒内说出它锁住了什么吗？
6. `arr[i]` 和 `*(arr+i)` 为什么完全等价？
7. “地址传递本质上也是值传递”，这句话怎么理解？
8. 冒泡排序内层循环为什么是 `j < n-i-1`？总共比较几次？
9. 为什么排序函数需要单独传入数组长度？
10. 本章知识以后会在哪些课程中再次出现？

知识点—题目对应表

下表说明每道正式练习检查的是哪个知识点。做错某题时，可根据“对应讲次”回到正文复习。

编号	核心知识点	对应讲次	A 组基础（1–25）	B 组提升（26–35）	C 组跨学科（36–38）
K1	内存与地址	第一讲	2，19	33	38
K2	定义、& 与 *	第二讲	1，3，11，16，21	26，27	36，37
K3	指针的大小	第三讲	4，15，17	29，33	—
K4	空指针	第四讲	5，12	—	—
K5	野指针与越界	第四、六讲	6，12，24	30	37
K6	const 修饰指针	第五讲	7，8，13，22	28，34	—
K7	数组与指针算术	第六讲	9，14，19，24	26，28，31，33	38
K8	值传递与地址传递	第七讲	10，23	27，34	—
K9	数组作函数参数	第八讲	15，18	29，34，35	—
K10	冒泡排序	第八讲	20，25	30，32，35	—

每个核心知识点至少被一道基础题覆盖；主要知识点在提升题中再次出现；3 道跨学科题分别借用**数据结构**（链表）、**嵌入式系统**（寄存器）、**图像处理**（像素数组）三个不同背景。

正式练习

全部练习共 **38 道**：A 组基础巩固 25 道、B 组中等偏下提升 10 道、C 组跨学科迁移 3 道。答案集中在练习之后，请先独立完成。除特别说明外，均假设为 **64 位程序**，`int` 占 4 字节，并已包含 `<iostream>` 与 `using namespace std;`。题图中的地址为示意地址。

A. 基础巩固训练（25 道）

A-1　单项选择题（10 道）

1. 基础　K2 取地址与解引用

执行 `int a = 10; int *p = &a;` 后，下列说法正确的是（　）



题图 1　两行代码执行后的内存

(A) `p` 的值是 10　　(B) `*p` 的值是 10

(C) `&p` 与 `&a` 相等　　(D) `*p` 的值是 `a` 的地址

2. 基础　K1 内存与地址

关于内存地址，下列说法正确的是（　）



题图 2　一个 `int` 变量在字节编号的内存中

(A) 内存中每个字节都有一个编号，即地址　　(B) 地址只能用十六进制表示

(C) 变量名就是变量的地址　　(D) `int` 变量占 4 字节，所以它有 4 个“变量地址”

3. 基础　K2 指针定义语法

要定义一个指向 `int` 的指针变量 `p`，下列写法正确的是（　）

(A) `int p*;`　　(B) `int *p;`

(C) `*int p;`　　(D) `int p = &a;`

4. 基础 K3 指针的大小

在 64 位程序中执行 `double d = 3.14; double *pd = &d;`，则 `sizeof(pd)` 的值为（ ）

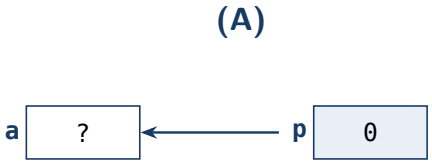


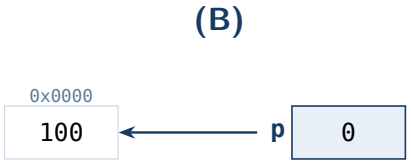
题图 4

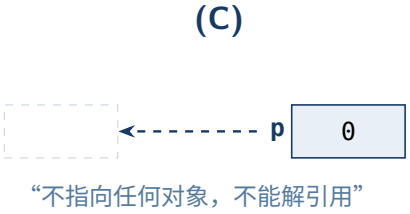
(A) 4
(B) 8
(C) 16
(D) 与 d 的值有关

5. 基础 K4 空指针

执行 `int *p = nullptr;` 后，下列说法正确的是（ ）。四个选项所描述的情形分别画在下面：





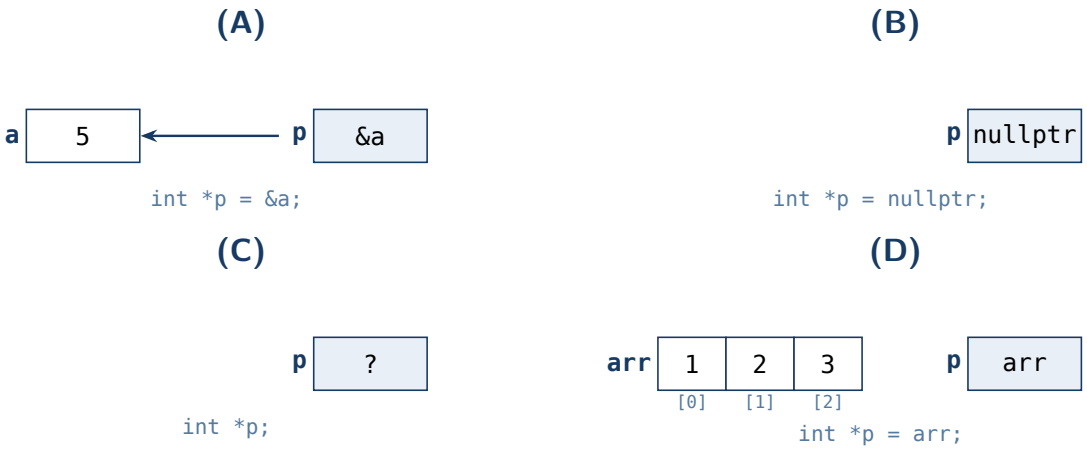




(A) p 指向某个 int 变量
(B) 可以执行 *p = 100;
(C) p 不指向任何对象，不能解引用
(D) p 占用 0 字节

6. 基础 K5 野指针

设 a 是已定义的 int 变量，arr 是已定义的 int 数组，下列均为**函数内的局部定义**。会使 p 成为野指针的是（ ）



(A) `int *p = &a;`

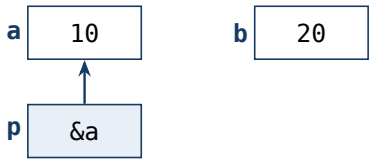
(B) `int *p = nullptr;`

(C) `int *p;`

(D) `int *p = arr;`

7. 基础 K6 const 修饰指针

设 `int a = 10, b = 20; const int *p = &a;`，下列语句能通过编译的是（ ）



题图 7 声明后的初始状态

(A) `*p = 20;`

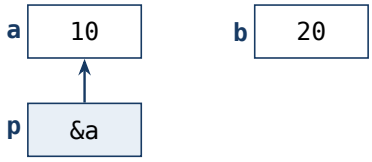
(B) `p = &b;`

(C) A、B 都可以

(D) A、B 都不可以

8. 基础 K6 const 修饰指针

设 `int a = 10, b = 20; int * const p = &a;`，下列语句能通过编译的是（ ）



题图 8 声明后的初始状态

(A) `p = &b;`

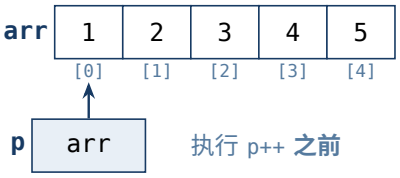
(B) `*p = 20;`

(C) `p++;`

(D) 以上都不能

9. 基础　K7 指针算术

执行 `int arr[] = {1, 2, 3, 4, 5}; int *p = arr; p++;` 后，`*p` 的值为（　）



题图 9

(A) 1　　(B) 2

(C) `arr` 的地址加 1　　(D) 5

10. 基础　K8 值传递

下列程序的输出是（　）

```
1 void swap(int a, int b) { int t = a; a = b; b = t; }
2 int main() {
3     int a = 10, b = 20;
4     swap(a, b);
5     cout << "a=" << a << " b=" << b << endl;
6 }
```



题图 10　调用前 main 中的变量

(A) `a=20 b=10`　　(B) `a=10 b=20`

(C) 编译错误　　(D) 输出不确定

A-2　多项选择题（5 道）

11. 基础　K2 & 与 *

设 `int a = 5; int *p = &a;`，下列表达式中值**等于** 5 的有（　）



题图 11

(A) `*p`　　(B) `a`

(C) `*&a`　　(D) `&*p`

12. 基础 K4 K5 空指针与野指针

关于空指针与野指针，下列说法正确的有（ ）



题图 12 简化的内存地图

(A) 空指针可以用来初始化指针变量

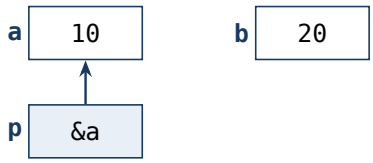
(B) 野指针指向的内存不是程序申请的空间

(C) 对空指针解引用是安全的，只是会得到 0

(D) 定义指针时若暂时无处可指，应初始化为 `nullptr`

13. 基础 K6 const 修饰指针

设 `int a = 10, b = 20; const int * const p = &a;`，下列语句中**不能**通过编译的有（ ）



题图 13

(A) `p = &b;`

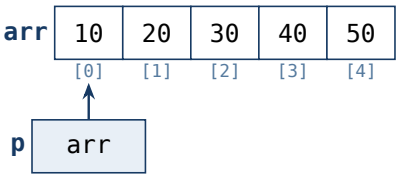
(B) `*p = 30;`

(C) `cout << *p;`

(D) `int x = *p + 1;`

14. 基础 K7 数组与指针

设 `int arr[5] = {10, 20, 30, 40, 50}; int *p = arr;`，下列表达式中值**等于 30** 的有（ ）



题图 14

(A) `*(p + 2)`

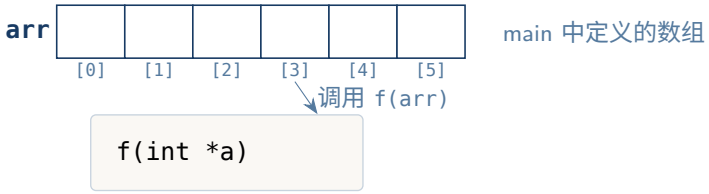
(B) `p[2]`

(C) `arr[2]`

(D) `*p + 2`

15. 基础 K3 K9 sizeof 与数组传参

设 64 位程序中有 `int arr[6];`，并有函数 `void f(int *a);`。下列说法正确的有（ ）



题图 15

(A) `sizeof(int*) == sizeof(char*)`

(B) `sizeof(p)` 的大小由 `p` 所指变量的值决定

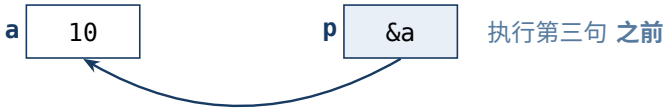
(C) 在 main 中 `sizeof(arr)/sizeof(arr[0])` 等于 6

(D) 在 `f` 中 `sizeof(a)/sizeof(a[0])` 仍等于 6

A-3 填空题（5 道）

16. 基础 K2 解引用修改变量

执行 `int a = 10; int *p = &a; *p = 100;` 后，`a` = _____，`*p` = _____。



题图 16

17. 基础 K3 指针的大小

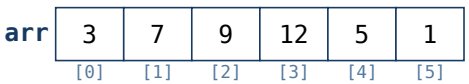
在 64 位程序中 `sizeof(int*)` = _____ 字节；在 32 位程序中 `sizeof(int*)` = _____ 字节。



题图 17 指针中保存的地址宽度

18. 基础 K9 sizeof 求数组长度

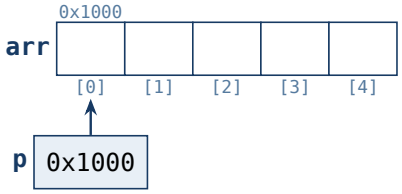
设 `int arr[] = {3, 7, 9, 12, 5, 1};`，则 `sizeof(arr)` = _____，`sizeof(arr)/sizeof(arr[0])` = _____。



题图 18

19. 基础 K1 K7 地址计算

设 `int arr[5];` 的首地址为 `0x1000`，`int *p = arr;`，则 `p + 3` 的值为 _____，`*(p + 3)` 就是 `arr[____]`。



题图 19　只标出了首元素地址

20. 基础 K10 冒泡排序循环

对 5 个元素做冒泡排序（外层 `i` 从 0 开始），外层最多 _____ 轮；当 `i = 1` 时内层比较 _____ 次；全部比较次数为 _____ 次。



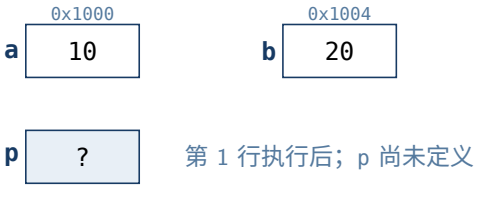
题图 20　5 个元素

A-4　解答题（5 道）

21. 基础 K2 定义、赋值与解引用

阅读程序，(1) 画出每条语句执行后的内存图；(2) 写出输出结果。

```
int a = 10, b = 20;
int *p = &a;
*p = 30;
p = &b;
*p = 40;
cout << a << " " << b << " " << *p << endl;
```



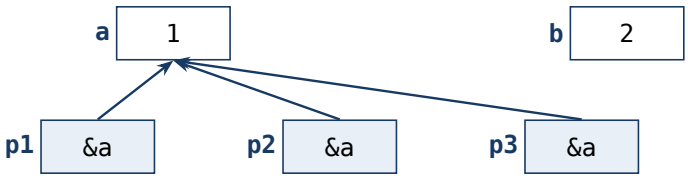
题图 21　起始状态

22. 基础 K6 const 三种情况

设 `int a = 1, b = 2;` 且

```
const int *p1 = &a;
int * const p2 = &a;
const int * const p3 = &a;
```

判断下列语句能否通过编译，并说明理由：① `*p1 = 5;` ② `p1 = &b;` ③ `*p2 = 5;` ④ `p2 = &b;` ⑤ `*p3 = 5;` ⑥ `a = 5;`



题图 22　三个指针都指向 a

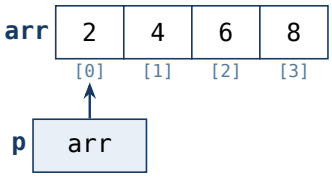
23. 基础 K8 值传递与地址传递

(1) 写出上面第 10 题程序的输出，并画图说明 `swap` 为什么没有交换 `main` 中的 `a`、`b`；(2) 把 `swap` 改写成地址传递版本 `swap_p`，写出调用语句，并说明它为什么能交换。

24. 基础 K5 K7 指针遍历与越界

阅读程序，回答：(1) 输出是什么？(2) 循环结束后 `p` 指向哪里？此时能否执行 `cout << *p;`？(3) 笔记原代码数组有 5 个元素却写 `i < 10`，有什么问题？应怎样修改？

```
int arr[] = {2, 4, 6, 8};
int *p = arr;
for (int i = 0; i < 4; i++) { cout << *p << " "; p++; }
```



题图 24　循环开始前

25. 基础 K10 冒泡排序过程

对数组 `{6, 2, 8, 4, 1}` 进行升序冒泡排序。(1) 写出第 1 轮（`i = 0`）中每一次比较后的数组状态；(2) 写出第 2 轮结束后的数组。



题图 25

B. 中等偏下提升训练（10 道）

B-1 单项选择题（3 道）

26. 提升 K7 K2 指针偏移

设 `int arr[] = {1, 2, 3, 4, 5}; int *p = arr + 1;`，则 `*(p + 2)` 的值为（ ）



题图 26

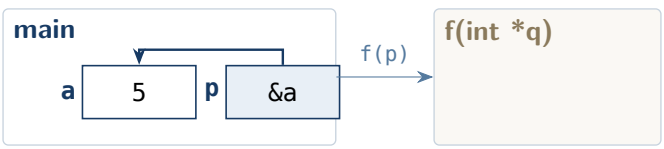
(A) 3　　(B) 4

(C) 5　　(D) 2

27. 提升 K8 K2 指针形参也是拷贝

下列程序执行后，`a` 的值和 `p` 的状态是（ ）

```
void f(int *q) {
    *q = *q * 2;
    q = nullptr;
}
int main() {
    int a = 5;
    int *p = &a;
    f(p);
}
```



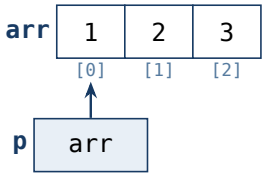
题图 27　调用前

(A) `a = 10`，`p` 变为空指针　　(B) `a = 10`，`p` 仍指向 `a`

(C) `a = 5`，`p` 变为空指针　　(D) `a = 5`，`p` 仍指向 `a`

28. 提升 K6 K7 常量指针在数组上

设 `int arr[] = {1, 2, 3}; const int *p = arr;`，下列代码能通过编译的是（ ）



题图 28

(A) `p++; cout << *p;`　　(B) `*p = 0;`

(C) `p[1] = 5;`　　(D) `*(p + 1) = 5;`

B-2 多项选择题（2 道）

29. 提升 K3 K9 数组传参后的 sizeof

64 位程序如下，下列说法正确的有（ ）

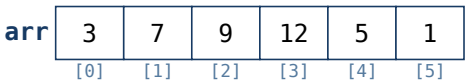
```
1 void show(int *a) { cout << sizeof(a) << endl; }
2 int main() {
3     int arr[10];
4     cout << sizeof(arr) << endl;
5     show(arr);
6 }
```

(A) main 中输出 40　　(B) show 中输出 8

(C) 在 show 中 `sizeof(a)/sizeof(a[0])` 等于 10　　(D) 因此排序函数通常需要额外传入长度 n

30. 提升 K10 K5 冒泡排序边界

笔记封装版冒泡排序的内层循环误写为 `for (int j = 0; j < n - j - 1; j++)`。对数组 `{3, 7, 9, 12, 5, 1}`（$n = 6$）调用该函数，下列说法正确的有（ ）



题图 30

(A) 输出仍为 `3 7 9 12 5 1`　　(B) 每一轮中 `j` 只能取 0、1、2

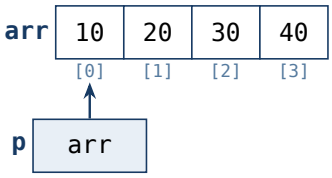
(C) 改为 `j < n - i - 1` 后输出 `1 3 5 7 9 12`　　(D) 该错误会导致 `arr[j+1]` 越界访问

B-3 填空题（3 道）

31. 提升 K7 指针与数组综合

执行下列代码后，数组 `arr` 为 {____,____,____,____}，此时 `p` 指向 `arr[____]`。

```
int arr[] = {10, 20, 30, 40};
int *p = arr;
*(p + 1) += 5;
p += 2;
*p = *(p - 1) + 1;
```



题图 31　第 2 行执行后

32. 提升 K10 比较与交换次数

对 6 个元素做冒泡排序（按笔记的两层循环，不提前结束）：总比较次数为 _____；若原数组已经升序，交换次数为 _____；若原数组为 `{6,5,4,3,2,1}`，交换次数为 _____。

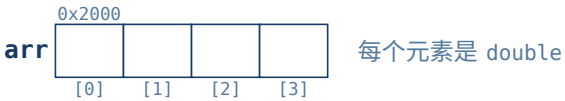


题图 32

33. 提升 K1 K3 K7 地址与步长

64 位程序中，`double arr[4];` 的首地址为 `0x2000`，`double *q = &arr[3];`。则 `q` 的值为 _____，`q - arr` = _____，`sizeof(q)` = _____。

（提示：指向同一数组的两个指针相减，结果是它们之间相隔的**元素个数**。）



题图 33

B-4 综合大题（2 道）

34. 提升 K6 K8 K9 用指针“返回”两个结果

一个函数只能 return 一个值。现在希望一次求出数组的最小值和最大值，设计函数

```
void minmax(const int *arr, int n, int *pmin, int *pmax);
```

(1) 为什么 pmin、pmax 要设计成指针？为什么 arr 前面加 const？

(2) 补全函数体。

(3) 在 main 中执行 int a[] = {4, 9, 1, 7}; int mn, mx; minmax(a, 4, &mn, &mx);，画出函数即将返回时的内存图，并写出 mn、mx 的值。



题图 34 调用前

35. 提升 K9 K10 排序函数的调试与改进

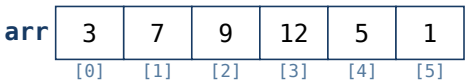
笔记中封装的冒泡排序如下（第 392 页原样）：

```
void BubbleSort(int *arr, int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - j - 1; j++)
            if arr[j] > arr[j+1]
            { int temp = arr[j]; arr[j] = arr[j+1]; arr[j+1] = temp; }
}
```

(1) 指出其中的两处错误并改正；

(2) 用改正后的函数对 {3, 7, 9, 12, 5, 1} 排序，写出每一轮结束后的数组；

(3) 若数组本来就是 {1, 2, 3, 4, 5, 6}，改正后的函数共比较几次？如果增加一个标志变量 swapped，在某一轮没有发生任何交换时立即结束，则需要比较几次？



题图 35

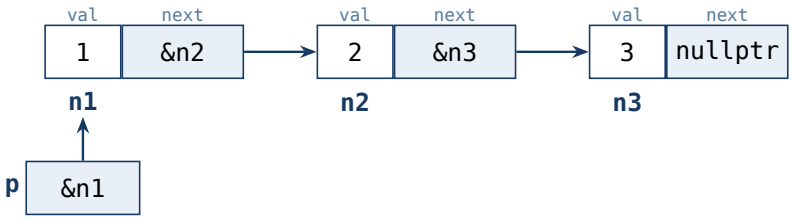
C. 跨学科迁移训练（3 道）

本组题目的背景来自其他课程，但**完成本题所需的背景信息都已在题干中给出**，真正考查的仍是本章知识。

36. 跨学科　K2 × 数据结构

背景（数据结构 · 链表）：链表把数据分散存放在若干“结点”中，每个结点除了保存数据 `val`，还保存下一个结点的地址 `next`；最后一个结点的 `next` 为 `nullptr`。代码中 `p->val` 等价于 `(*p).val`，意思是“先解引用 `p` 找到结点，再取它的 `val`”。

```
struct Node { int val; Node *next; };
Node n3 = {3, nullptr};
Node n2 = {2, &n3};
Node n1 = {1, &n2};
Node *p = &n1;
p = p->next;
cout << p->val << endl;
```



题图 36　三个结点组成的链表（执行 `p = p->next` 之前）

输出为（　）

(A) 1　　(B) 2

(C) 3　　(D) 编译错误

37. 跨学科　K2 K5 × 嵌入式系统

背景（嵌入式 · 寄存器）：某单片机的芯片手册规定，地址 `0x40020014` 处是控制 LED 引脚的“输出寄存器”，向它写入 1 可以点亮 LED。程序如下（`volatile` 的作用是告诉编译器每次都真实地读写这个地址，本题可以忽略它）：

```
volatile unsigned int *reg = (volatile unsigned int *)0x40020014;
*reg = 1;      // 点亮 LED
```



题图 37

下列说法正确的有（ ）

(A) 这里把整数地址强制转换为指针，与笔记中 `(int*)0x0010` 的写法形式相同

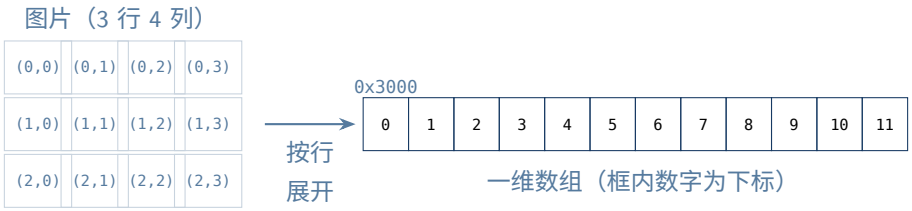
(B) 在普通电脑程序中，随便写一个地址这样访问同样是安全的

(C) 能否访问，取决于这个地址是否是程序可以合法访问的空间（这里由芯片手册保证）

(D) `*reg = 1` 是通过解引用向该地址写入数据

38. 跨学科 K1 K7 × 图像处理

背景（图像处理 · 像素数组）：一张宽 $W = 4$、高 $H = 3$ 的灰度图片，每个像素用 1 个字节（`unsigned char`）表示亮度，所有像素**按行依次**存放在一个一维数组中，第 r 行第 c 列（r, c 都从 0 开始）的像素下标为 $r \times W + c$。



题图 38

设 `unsigned char *img` 指向下标 0 的像素，地址为 `0x3000`。第 2 行第 1 列的像素下标为 _____，地址为 _____，可以写成 `*(img +_____)`。若改为每个像素用 4 字节的 `int` 存放，`int *q` 指向 `0x3000`，则同一个像素的地址为 _____。

全部答案速查

只列结果，过程见后面的详细解析。所有涉及代码的答案均已用 g++（C++17，64 位 Linux）编译或运行核对。

A 组基础巩固

单选 1–10	1. B　2. A　3. B　4. B　5. C　6. C　7. B　8. B　9. B　10. B
多选 11–15	11. ABC　12. ABD　13. AB　14. ABC　15. AC
填空 16–20	16. 100，100　17. 8，4　18. 24，6　19. `0x100C`，3　20. 4，3，10
解答 21–25	21. 输出 `30 40 40`　22. ①✗ ②✓ ③✓ ④✗ ⑤✗ ⑥✓　23. 输出 `a=10 b=20`；改为 `swap_p(&a,&b)`　24. `2 4 6 8`；`p` 指向 `arr+4`，不能解引用；改为 `i < len`　25. 第 1 轮末 `2 6 4 1 8`；第 2 轮末 `2 4 1 6 8`

B 组中等偏下提升

单选 26–28	26. B　27. B　28. A
多选 29–30	29. ABD　30. ABC
填空 31–33	31. `{10,25,26,40}`，2　32. 15，0，15　33. `0x2018`，3，8
综合 34–35	34. `mn = 1`，`mx = 9`　35. 改 `n-j-1`→`n-i-1`、补 `if` 括号；各轮后依次为 [3 7 9 5 1 12] [3 7 5 1 9 12] [3 5 1 7 9 12] [3 1 5 7 9 12] [1 3 5 7 9 12]；比较 15 次，加标志后 5 次

C 组跨学科迁移

36–38	36. B　37. ACD　38. 9，`0x3009`，9，`0x3024`

A 组详细解析

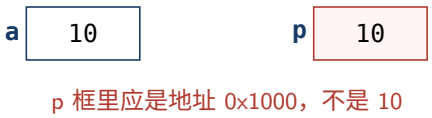
A-1　单项选择题

第 1 题　基础　　答案：**B**　　考点：K2 取地址与解引用（第二讲）

对应知识　p = &a 时 *p ≡ a；p 的值是地址。

思路　先分清图上的四个量：a（框内 10）、&a（0x1000）、p（框内 0x1000）、&p（0x2000），再逐项对照。

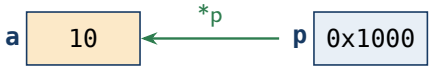
(A) ✗ 错误



p 框里应是地址 0x1000，不是 10

p 是指针，保存的是 a 的**地址**，不是 a 的值。

(B) ✓ 正确



沿箭头解引用到达 a，*p 就是 a，值为 10。

(C) ✗ 错误



两个方框上方的地址不同

&p 是 p 自己的地址（0x2000），&a 是 0x1000，二者是两个不同变量的地址。

(D) ✗ 错误



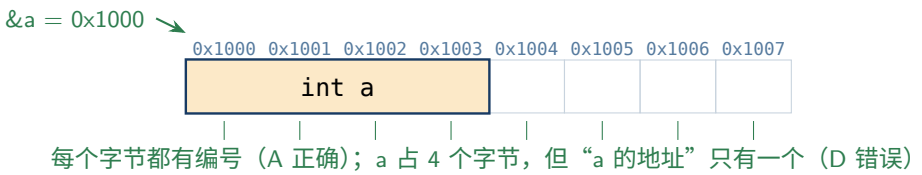
“a 的地址”是 p 的值，不是 *p 的值

把 p 与 *p 混淆了：a 的地址是 p 的值。

常见错误　把“箭头起点”（p）和“箭头终点”（*p）搞反。

一句话总结　p 是门牌号，*p 是那户人家。

第 2 题　基础　　答案：**A**　　考点：K1 内存与地址（第一讲）



解析图 2

逐项　(A) 正确，这正是地址的定义。(B) 错误：十六进制只是书写习惯，地址本质是整数，也可以用十进制表示。(C) 错误：变量名是源代码中的标签，地址要用 &a 获得。(D) 错误：变量占 4 个字节（有 4 个字节编号），但**变量的地址**规定为首字节编号，只有一个（见解析图 2 绿色箭头）。

一句话总结　每个字节一个编号，每个变量一个地址（首字节）。

第 3 题 基础　答案：B　　考点：K2 指针定义语法（第二讲）

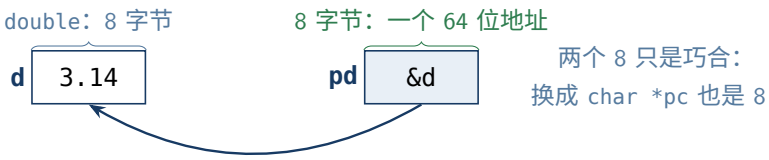
可视化判定 本题考查纯语法记忆，画图不能降低难度，故不配图。

判断 语法为“数据类型 * 指针变量名”，只有 (B) `int *p;` 符合。(A) * 位置错误；(C) `*int` 不是合法类型写法；(D) `int p` 是普通整数，不能保存地址（`&a` 的类型是 `int*`，编译报错）。

常见错误 写 `int* p, q;` 以为定义了两个指针（见第二讲易错点）。

一句话总结 * 紧贴变量名：`int *p`。

第 4 题 基础　答案：B　　考点：K3 指针的大小（第三讲）



解析图 4

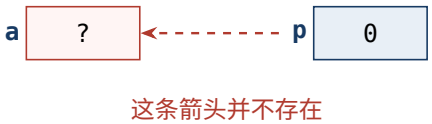
判断 64 位程序中任何数据指针都是 8 字节（第三讲推导：$64 \div 8 = 8$），选 (B)。(A) 是 32 位程序的结果；(C) 无依据；(D) 指针大小与所指对象的值、甚至类型都无关。

常见错误 看到 `double` 是 8 字节，就以为“`double*` 是 8 字节是因为 `double` 是 8 字节”。

一句话总结 指针多大只看地址多宽。

第 5 题 基础　答案：C　　考点：K4 空指针（第四讲）

(A) ✗ 错误



空指针的定义就是“不指向任何对象”。

(B) ✗ 错误



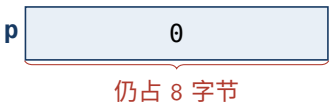
解引用空指针是未定义行为，常见系统上会段错误。

(C) ✓ 正确



正是空指针的定义与注意事项（笔记 §7.4）。

(D) ✗ 错误



p 的**值**为空，但 p 这个变量本身照样占 8 字节。

常见错误 把“值为空”误解为“不占空间”；或以为 *p 会读出 0。

一句话总结 空指针：有变量、有值（空），没有目标。

第 6 题　基础　答案：C　　考点：K5 野指针（第四讲）

(A) ✗ 错误



指向已定义的变量，合法。

(B) ✗ 错误



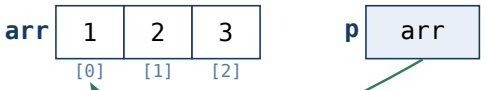
空指针不是野指针：它的值是确定的、可检查的。

(C) ✓ 正确



局部指针未初始化，值不确定，指向未知位置——典型野指针。

(D) ✗ 错误



数组名转换为首元素地址，指向合法数组元素。

补充　笔记中的另一个来源 `int *p = (int*)0x0010;` 也是野指针，此外还有越界。

一句话总结　“定义了却没给值”的指针就是野的。

第 7 题　基础　答案：B　　考点：K6 const 修饰指针（第五讲）

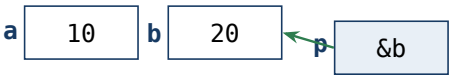
(A) ✗ 错误



const 在 * 左：值被锁

`*p` 的类型是 `const int`，不能赋值。

(B) ✓ 正确



p 本身没被锁，可改指向

`p` 的类型是 `const int*`，最外层无 const，可以改指向。

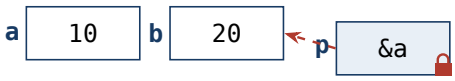
C、D　由上，只有 B 合法，所以 C、D 都错。

三步法　找 * → const 在左 → 锁值；A 改的是值（✗），B 改的是指向（✓）。

一句话总结　const 在 * 左边：锁值不锁指向。

第 8 题　基础　答案：B　　考点：K6 const 修饰指针（第五讲）

(A) ✗ 错误



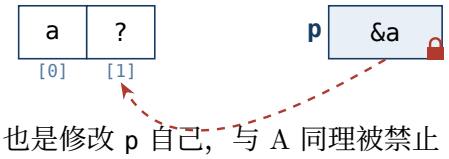
const 在 * 右边，p 本身是常量，不能改指向。

(B) ✓ 正确



*p 类型为普通 int，可以赋值。

(C) ✗ 错误



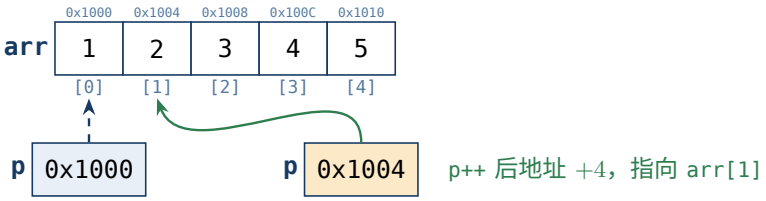
p++ 也是修改 p 自己，与 A 同理被禁止（g++ 实测报错）。

(D) ✗ 错误

B 合法，所以“都不能”错误

一句话总结 const 在 * 右边：锁指向不锁值；p++、p = ... 都算改指向。

第 9 题　基础　答案：B　　考点：K7 指针算术（第六讲）



解析图 9　虚线：执行前；绿色：执行后

判断　p++ 使 p 前进一个 int（4 字节），指向 arr[1]，*p = 2，选 (B)。(A) 是 p++ 之前的值；(C) 混淆了“指针值”与“解引用的值”，且步长也不是 1；(D) 是最后一个元素。

一句话总结 p++ 走到“下一个元素”。

第 10 题　基础　答案：B　　考点：K8 值传递（第七讲）



解析图 10

判断　形参是实参的拷贝，交换的是副本（第七讲定理），输出 a=10 b=20，选 (B)。程序语法正确，故 C 错；行为确定，故 D 错。

一句话总结 值传递改不了实参。

A-2　多项选择题

第 11 题　基础　答案：**ABC**　　考点：K2 & 与 *（第二讲）

(A) ✓ 正确



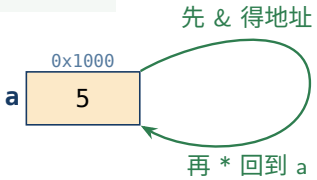
*p：从 p 顺箭头到 a，值 5。

(B) ✓ 正确



a 本身就是 5。

(C) ✓ 正确



*&a ≡ a（第二讲三条关系之一）。

(D) ✗ 错误

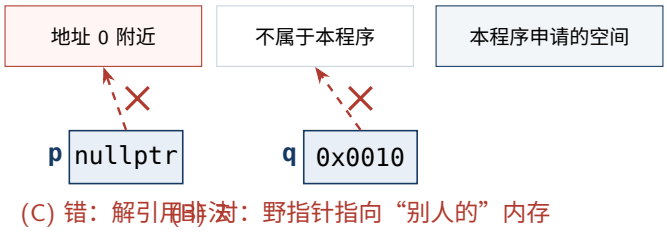


&*p = p，值是地址 0x1000，不是 5。

正确答案　ABC。

一句话总结　* 和 & 相互抵消：*&a 是变量，&*p 是地址。

第 12 题　基础　答案：**ABD**　　考点：K4 K5 空指针与野指针（第四讲）



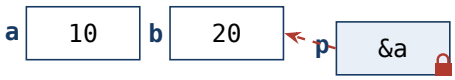
解析图 12

逐项　(A) 正确：笔记“用途：初始化指针变量”。(B) 正确：笔记红字总结。(C) 错误：解引用空指针是未定义行为，不会“安全地得到 0”（解析图中红叉）。(D) 正确：推荐的良好习惯。

一句话总结　空指针用来“占位”和“检查”，不用来访问。

第 13 题　基础　答案：AB　　考点：K6 const 修饰指针（第五讲）

(A) ✓ 正确



不能通过编译（右边有 const，锁指向）。

(B) ✓ 正确



不能通过编译（左边有 const，锁值）。

(C) ✗ 错误



只是读取 *p，可以编译。本题问“不能”，故不选。

(D) ✗ 错误



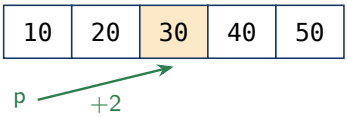
同样只读，x 为 11，可以编译。

正确答案　AB（题目问“不能通过编译”）。

一句话总结　const 只禁止“写”，从不禁止“读”。

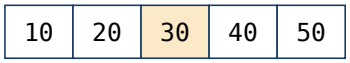
第 14 题　基础　答案：ABC　　考点：K7 数组与指针（第六讲）

(A) ✓ 正确



p+2 指向 arr[2]，解引用得 30。

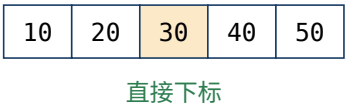
(B) ✓ 正确



p[2] ≡ *(p+2)

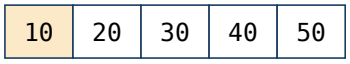
下标运算的定义，与 A 相同。

(C) ✓ 正确



arr[2] = 30。

(D) ✗ 错误

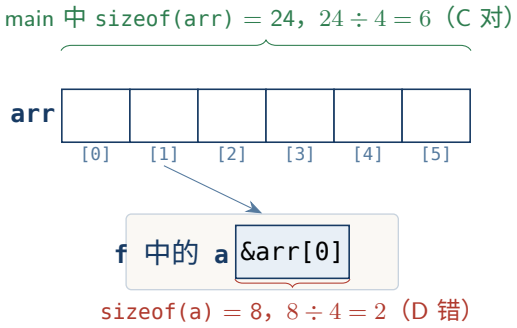


先取 *p = 10，再 +2 = 12

* 的优先级高于 +：*p + 2 = 12。

一句话总结　括号决定先移动还是先取值：*(p+2) ≠ *p+2。

第 15 题　基础　答案：AC　　考点：K3 K9 sizeof 与数组传参（第三、八讲）



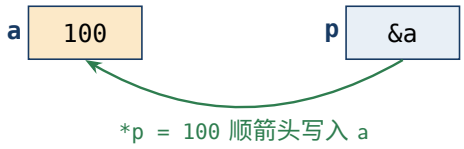
解析图 15

逐项 (A) 正确：所有指针同样大（8 字节）。(B) 错误：指针大小与所指的值无关。(C) 正确：在定义数组处，`sizeof(arr)` 为整个数组字节数。(D) 错误：形参 `a` 是指针，`sizeof(a)` = 8，结果为 2。

一句话总结 数组一旦传进函数，就只剩一个指针。

A-3 填空题

第 16 题 基础 答案：**100；100** 考点：K2 解引用修改变量



解析图 16

推理 `*p` ≡ `a`，所以 `*p = 100` 就是 `a = 100`；之后 `*p` 读的仍是 `a`，也是 100。

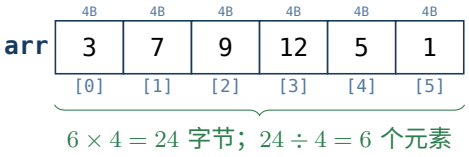
易错 以为只有 `*p` 变了、`a` 没变——它们是同一个变量。

第 17 题 基础 答案：**8；4** 考点：K3 指针的大小

推理 64 位地址 = 64 ÷ 8 = 8 字节；32 位地址 = 32 ÷ 8 = 4 字节（第三讲推导）。对照题图 17：长条的宽度就是答案。

易错 “32/64 位”指程序的目标平台，而非操作系统（原图纠错 ⑥）。

第 18 题 基础 答案：**24；6** 考点：K9 sizeof 求数组长度

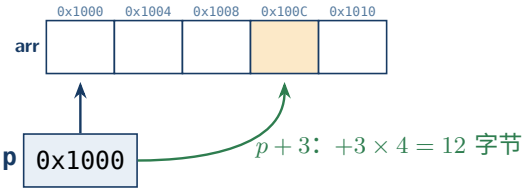


解析图 18

推理 6 个 `int`，每个 4 字节，`sizeof(arr)` = 24；`sizeof(arr[0])` = 4，相除得 6。

等价答案 “24 字节”“6 个”均可。

第 19 题 基础　答案：**0x100C; 3**　　考点：K1 K7 地址计算

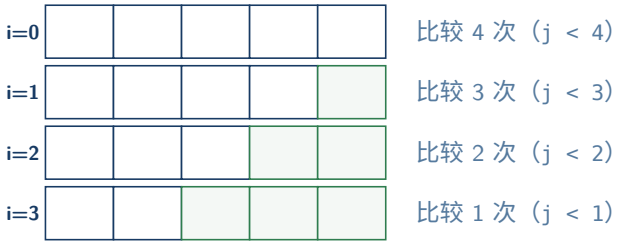


解析图 19

公式　地址 $(p+3)=$ `0x1000` $+3\times$ `sizeof(int)` $=$ `0x1000` $+12=$ `0x100C`（十进制 $12=$ 十六进制 C）；`*(p+3)` $\equiv$ `arr[3]`。

易错　写成 `0x1003`（忘记乘 4）或 `0x1012`（把十进制 12 直接接在后面）。

第 20 题 基础　答案：**4; 3; 10**　　考点：K10 冒泡排序循环



解析图 20　绿色为已就位的元素

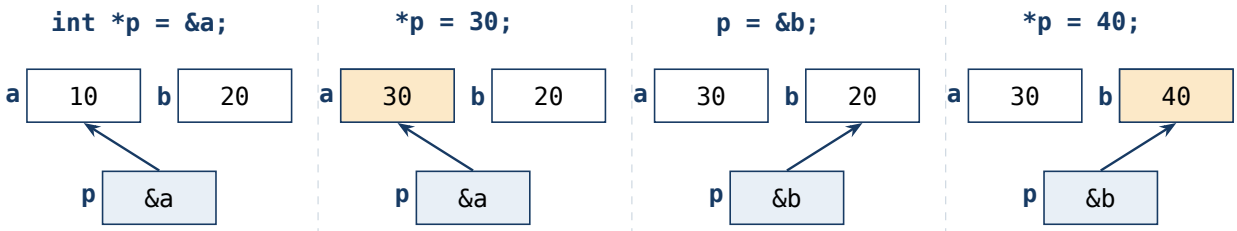
推理　外层 `i < n-1` $=4$ 轮；`i=1` 时内层 `j < 5-1-1 = 3`，比较 3 次；总数 $4+3+2+1=10=\frac{5\times4}{2}$。

A-4　解答题

第 21 题 基础　答案：**输出 30 40 40**　　考点：K2 定义、赋值与解引用

题意分析　指针先指向 a，改一次；再改指向 b，再改一次。关键是每次 *p 此刻指向谁。

使用关系　p = &x 时 *p $\equiv$ x（第二讲）。



解析图 21　第 2–5 行执行后的四个状态（黄色为刚被修改的变量）

第一步　第 3 行：p 指向 a，*p = 30 使 a = 30。

第二步　第 4 行：p 改为指向 b，a、b 都不变。

第三步　第 5 行：`*p = 40` 使 `b = 40`。最后 `*p` 即 `b`。

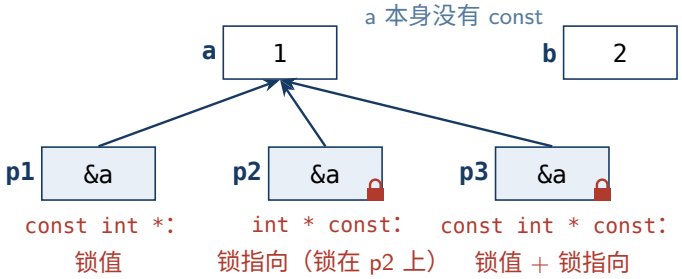
结果　输出 `30 40 40`（已运行核对）。

检查　`a` 只被改过一次（30），`b` 只被改过一次（40），与图一致。

易错点　以为第 4 行会把 `a` 的值“带到”`b`——改指向不会改任何变量的值。

同类题以后怎么做　每执行一行，就在图上更新“箭头”或“箭头终点的值”，二者只会变一个。

第 22 题　基础　答案：①✗ ②✓ ③✓ ④✗ ⑤✗ ⑥✓　考点：K6 const 三种情况



解析图 22　锁画在指针上表示“指针不可改”；p1、p3 的“值锁”只作用于通过它们的访问

序号	语句	改的是	对应一侧有无 const	结论
①	`*p1 = 5;`	值	有（* 左）	✗ 编译错误
②	`p1 = &b;`	指向	无	✓
③	`*p2 = 5;`	值	无	✓
④	`p2 = &b;`	指向	有（* 右）	✗ 编译错误
⑤	`*p3 = 5;`	值	有	✗ 编译错误
⑥	`a = 5;`	变量 a 本身	a 未声明为 const	✓

说明　六条结论均经 g++ 逐条编译验证。⑥ 是第五讲“边界情况 (1)”：常量指针只限制“通过指针修改”，`a` 自己仍可改。

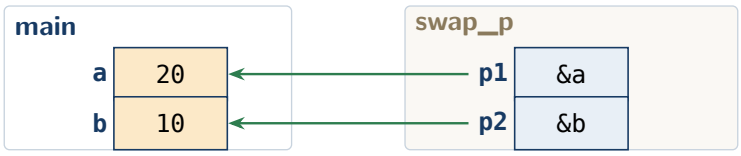
同类题以后怎么做　用“三步法”：找 * → 看 const 在哪侧 → 看语句改哪侧。

第 23 题　基础　答案：`a=10 b=20; swap_p(&a, &b)`　考点：K8 值传递与地址传递

(1)　输出 `a=10 b=20`。原因见解析图 10：形参是新变量，函数只交换了副本，`main` 中的变量从未被写（第七讲定理及证明）。

(2) 改写

```
void swap_p(int *p1, int *p2) { int temp = *p1; *p1 = *p2; *p2 = temp; }
// 调用：swap_p(&a, &b);   输出 a=20 b=10
```

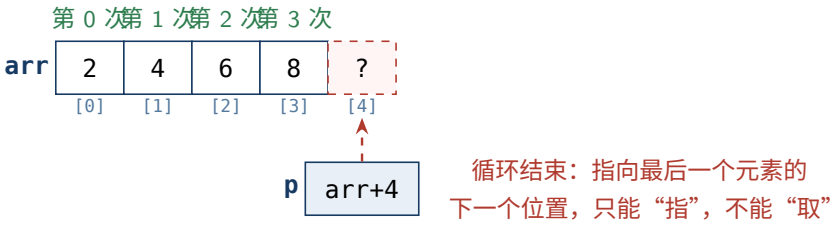


解析图 23　形参保存的是地址，*p1、*p2 就是 main 的 a、b

为什么能交换　传入的是地址的拷贝，拷贝来的地址仍指向原变量，由 *p ≡ x，对 *p1、*p2 赋值就是对 a、b 赋值。

易错点　调用时忘写 &（编译错误），或在函数中交换 p1、p2 本身（无效）。

第 24 题　基础　答案：**2 4 6 8；越界**　　考点：K5 K7 指针遍历与越界



解析图 24

(1)　每次输出 *p 再后移，依次输出 2 4 6 8（已运行核对）。

(2)　共执行 4 次 p++，p = arr + 4，即最后一个元素的下一个位置。它可以作为“终点标记”比较，但**不能**解引用，cout << *p 是越界读取（未定义行为）。

(3)　数组 5 个元素却循环 10 次，后 5 次读取数组以外的内存（原图纠错 ①）。改为 int len = sizeof(arr)/sizeof(arr[0]); 并用 i < len。

同类题以后怎么做　遍历前先问：“一共几个元素?”循环次数必须等于它。

第 25 题　基础　答案：**第 1 轮末 2 6 4 1 8；第 2 轮末 2 4 1 6 8**　　考点：K10 冒泡排序过程



解析图 25　黄色为下一次要比较的一对，绿色为已就位

(1) 各次比较后：`2 6 8 4 1`，`2 6 8 4 1`，`2 6 4 8 1`，`2 6 4 1 8`。

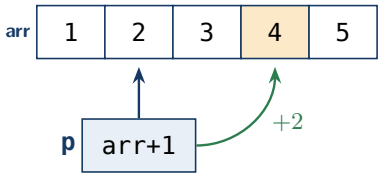
(2) 第 2 轮（`j = 0,1,2`）后：`2 4 1 6 8`。以上均由程序实际运行核对。

结果检查 每轮末尾就位的元素依次是 8、6，恰为最大、次大，符合第八讲的正确性定理。

B 组详细解析

B-1 单项选择题

第 26 题 提升　答案：**B**　　考点：K7 K2 指针偏移



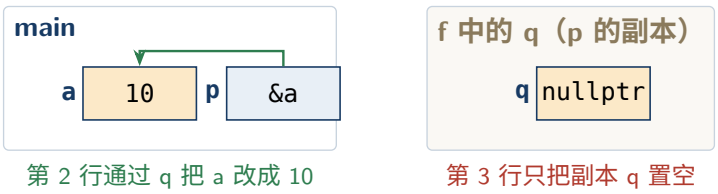
解析图 26　两次偏移：先 +1 到 arr[1]，再 +2 到 arr[3]

知识链 数组名 → 首元素地址 → 指针算术 → 解引用。`p = arr+1` 指向 `arr[1]`；`p+2 = arr+3`，`*(p+2) = arr[3] = 4`。

逐项 (A) 3 是 `*(arr+2)`，漏算了 `p` 的初始偏移；(C) 5 多算一格；(D) 2 是 `*p`。

一句话总结 偏移可以叠加：`(arr+1)+2 = arr+3`。

第 27 题 提升　答案：**B**　　考点：K8 K2 指针形参也是拷贝



解析图 27　函数结束时的状态

第一步为什么 调用 `f(p)` 时，`q` 是 `p` 的拷贝，值同为 `&a`（指针也是值传递）。

第二步 `*q = *q * 2` 顺箭头修改 `a`：$5 \times 2 = 10$。

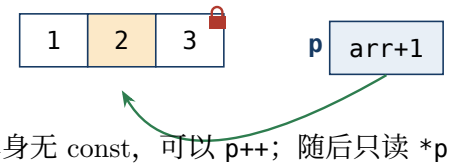
第三步 `q = nullptr` 只修改副本 `q`，`main` 中的 `p` 不受影响，仍指向 `a`。

逐项 (A) 错在以为改 `q` 会影响 `p`；(C) 两处都错；(D) 忽略了 `*q` 修改的是 `a`。已运行核对：`a=10`，`p==&a` 为真。

一句话总结 改 `*q` 影响外部，改 `q` 不影响外部。

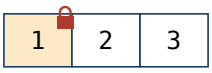
第 28 题 提升　答案：**A**　　考点：K6 K7 常量指针在数组上

(A) ✓ 正确



p 本身无 const，可以 p++；随后只读 *p（值 2），合法。

(B) ✗ 错误



写 *p：被锁

通过常量指针写，编译错误。

(C) ✗ 错误



p[1] 即 *(p+1)：被锁

下标写法本质也是 *(p+1)，同样禁止写。

(D) ✗ 错误



同 C

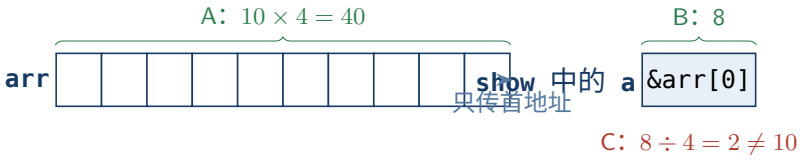
与 C 完全等价，编译错误。

说明　四个选项均经 g++ 编译核对：仅 A 通过。

一句话总结　const int *p 在数组上：可以“走”，只能“看”。

B-2　多项选择题

第 29 题 提升　答案：**ABD**　　考点：K3 K9 数组传参后的 sizeof

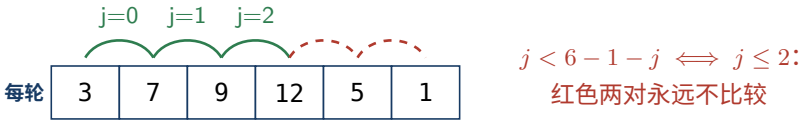


解析图 29

逐项　(A) 正确，整个数组 40 字节（已运行：输出 40）。(B) 正确，形参是指针，输出 8。(C) 错误，结果为 2。(D) 正确，这正是笔记中 BubbleSort(int *arr, int n) 需要参数 n 的原因。

哪个条件导致成立/不成立　关键条件是“sizeof 作用在数组上还是指针上”：同一个名字 arr，在 main 中是数组，在函数中已是指针。

第 30 题 提升　答案：**ABC**　　考点：K10 K5 冒泡排序边界



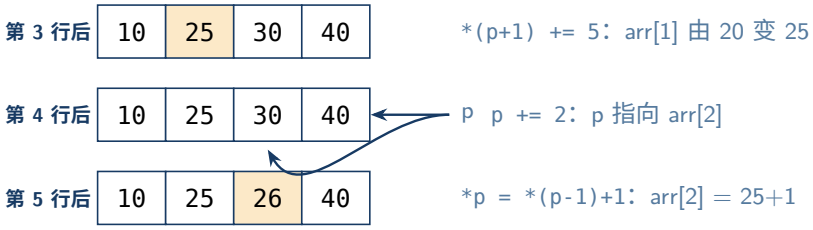
解析图 30　与第八讲图 8.4 相同的机制

逐项 (B) 正确：条件 $j < n-j-1$ 即 $2j<5$，$j\in\{0,1,2\}$，与 i 无关。(A) 正确：前 4 个数 3、7、9、12 本已递增，比较的 3 对都不交换，数组不变（已运行核对）。(C) 正确：改正后输出 `1 3 5 7 9 12`。(D) 错误：j 最大为 2，`arr[j+1]` 最远到 `arr[3]`，没有越界——这个错误的危害是“排不好”，而不是“越界”。

一句话总结 循环边界写错，可能不报错也不崩溃，只是结果悄悄错了。

B-3 填空题

第 31 题 提升　答案：`{10, 25, 26, 40}; 2`　　考点：K7 指针与数组综合



解析图 31

推理 第 3 行 `arr[1] = 25`；第 4 行 `p` 指向 `arr[2]`；第 5 行 `*(p-1)` 即 `arr[1]` $=25$，`arr[2] = 26`。最终 `{10,25,26,40}`，`p` 指向 `arr[2]`（已运行核对）。

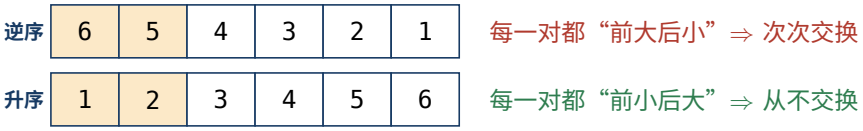
易错 第 5 行误用原来的 20，得到 21——必须使用第 3 行修改后的值。

第 32 题 提升　答案：`15; 0; 15`　　考点：K10 比较与交换次数

比较次数 按笔记的写法，比较次数只与 n 有关：$\frac{6\times5}{2}=15$，与数据无关（图 8.3）。

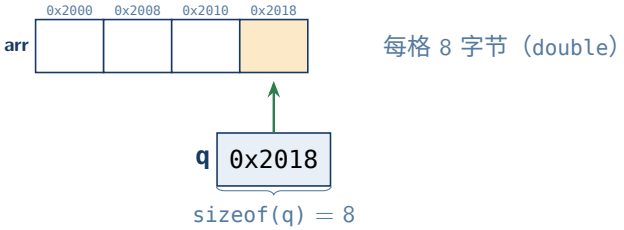
升序时 每次比较都满足 `arr[j] <= arr[j+1]`，一次也不交换：0。

逆序时 每次比较都满足 `arr[j] > arr[j+1]`（因为每轮参与比较的部分始终保持逆序），每次都交换：15。以上三个数均由程序计数核对。



解析图 32

第 33 题 提升　答案：`0x2018; 3; 8`　　考点：K1 K3 K7 地址与步长



解析图 33

推理　`&arr[3]` $=$ `0x2000` $+ 3 \times 8 =$ `0x2000` $+ 24 =$ `0x2018`（$24 = 0\text{x}18$）；`q - arr` 是相隔的元素个数 3（不是 24 字节）；`q` 是指针，64 位下 8 字节。

易错　按 `int` 的 4 字节计算得到 `0x200C`；或把 `q - arr` 写成 24。

B-4　综合大题

第 34 题 提升　答案：`mn = 1, mx = 9`　　考点：K6 K8 K9 用指针“返回”两个结果

1. 题目到底在问什么　需要一个函数把**两个**结果交给调用者，而 `return` 只能带回一个。

2. 已知条件整理　数组 `{4,9,1,7}`、长度 4；`mn`、`mx` 在 main 中，调用前未赋值。

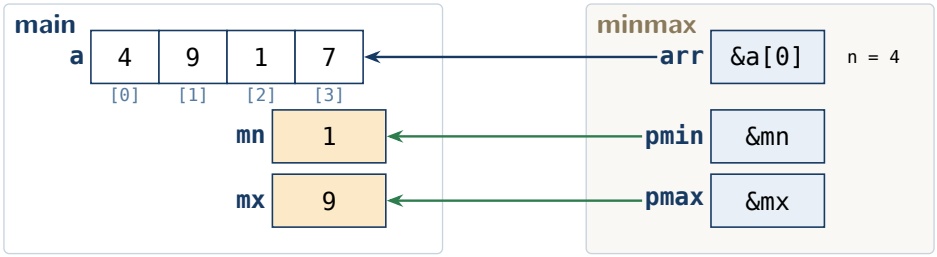
3. 知识链　地址传递可修改实参（第七讲）→ 数组传参只传首地址、需另传 `n`（第八讲）→ const 保护只读数据（第五讲）。

4. 第 (1) 问　`pmin`、`pmax` 是 main 中 `mn`、`mx` 的地址，函数通过 `*pmin = ...` 直接写入调用者的变量，相当于“带回”了两个结果；若用普通 `int`，只能改副本。`arr` 加 `const` 是因为函数只需读数组，加上后若误写 `arr[i] = ...` 编译器会报错。

5. 第 (2) 问

```
void minmax(const int *arr, int n, int *pmin, int *pmax)
{
    *pmin = arr[0];                 // 先假设第一个元素既是最小也是最大
    *pmax = arr[0];
    for (int i = 1; i < n; i++)
    {
        if (arr[i] < *pmin) *pmin = arr[i];
        if (arr[i] > *pmax) *pmax = arr[i];
    }
}
```

6. 第 (3) 问



解析图 34　函数即将返回时：三个指针形参分别指向 main 中的数组、mn、mx

循环过程：初值 `*pmin = *pmax = 4`；$i=1$：$9>4$，`*pmax = 9`；$i=2$：$1<4$，`*pmin = 1`；$i=3$：7 不改变。所以 `mn = 1`，`mx = 9`（已运行核对）。

7. 结果合理性检查　1 和 9 确实是数组中的最小、最大值；原数组未被修改（const 保证）。

8. 最容易卡在哪里　调用时写成 `minmax(a, 4, mn, mx)`（漏 &），或函数内写成 `pmin = arr[i]`（漏 *）。

9. 如果条件变化　若数组为空（$n=0$），`arr[0]` 越界——实际应用中需先判断 `n > 0`。

变式思考　如果还想“返回”平均值，应再加一个什么类型的参数？（`double *pavg`）

第 35 题　提升　答案：两处改正；15 次 / 5 次　考点：K9 K10 排序函数的调试与改进

1. 题目到底在问什么　找错—验证—优化三步。

2. 知识链　循环边界推导（第八讲）→ 正确性定理 → 比较次数公式。

3. 第 (1) 问　① 内层条件 `j < n - j - 1` 应为 `j < n - i - 1`（第八讲推导：第 i 轮末尾 i 个已就位）；② `if arr[j] > arr[j+1]` 缺少括号，应为 `if (arr[j] > arr[j+1])`，否则无法编译。

4. 第 (2) 问　每轮结束后的数组（程序运行核对）：

初始	3	7	9	12	5	1	
第 1 轮	3	7	9	5	1	12	比较 5 次
第 2 轮	3	7	5	1	9	12	比较 4 次
第 3 轮	3	5	1	7	9	12	比较 3 次
第 4 轮	3	1	5	7	9	12	比较 2 次
第 5 轮	1	3	5	7	9	12	比较 1 次

解析图 35　绿色区域每轮向左扩展一格

5. 第 (3) 问　改正后的函数比较次数只与 n 有关：$\frac{6\times5}{2}=15$ 次，即使数组本来有序。加标志的写法：

```
for (int i = 0; i < n - 1; i++)
{
    bool swapped = false;
```

```
    for (int j = 0; j < n - i - 1; j++)
        if (arr[j] > arr[j + 1])
        { int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp; swapped = true; }
    if (!swapped) break;        // 本轮没有交换，说明已经有序
}
```

对 {1,2,3,4,5,6}：第 1 轮比较 5 次、没有交换，立即结束，共 **5 次**（程序计数核对）。

6. 为什么“一轮不交换”就说明有序　一轮中每一对相邻元素都满足 arr[j] <= arr[j+1]，由不等式的传递性，整个参与部分已经递增；而末尾部分由正确性定理已就位且不小于前面，所以整个数组有序。

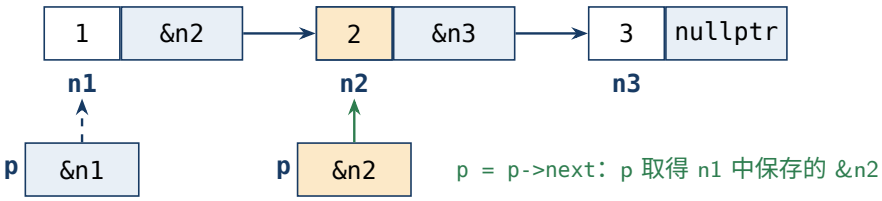
7. 结果合理性检查　第 5 轮后与 std::sort 结果一致。

8. 最容易卡在哪里　第 (2) 问中把“一次比较后的状态”和“一轮结束后的状态”混在一起。

变式思考　若要降序排列，只需改动哪一个符号？（把 > 改为 <。）

C 组详细解析（跨学科迁移）

第 36 题　跨学科　答案：B　　考点：K2 × 数据结构



解析图 36　虚线为执行前，绿色为执行后

推理　p->next 即 (*p).next：解引用 p 得到 n1，取其 next，值为 &n2；赋给 p 后 p 指向 n2，p->val = 2（已运行核对）。

逐项　(A) 是执行前的结果；(C) 需要再走一步；(D) 代码合法。

当前专题知识　解引用、“指针保存地址并可被重新赋值”（第二讲）。

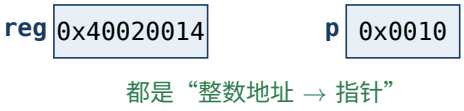
借用的外部情境　数据结构课程中的单向链表。

为什么不算超纲　题干给出了结点的含义和 -> 的等价写法，不需要预先学过结构体或链表。

迁移点　“p = &b 让指针改指向”这一关系，被搬到了“沿着链表走一步”的场景中：链表遍历就是反复执行 p = p->next。

第 37 题　跨学科　答案：ACD　　考点：K2 K5 × 嵌入式系统

(A) ✓ 正确



都是“整数地址 → 指针”

形式与笔记 `(int*)0x0010` 完全相同。

(B) ✗ 错误



在电脑程序中这是野指针，访问是未定义行为。

(C) ✓ 正确



合法与否不看写法，看地址是否属于可合法访问的空间。

(D) ✓ 正确



对 reg 解引用并赋值，把 1 写进该地址。

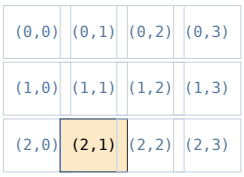
当前专题知识　强制类型转换得到的指针、解引用写内存、野指针的本质（第二、四讲）。

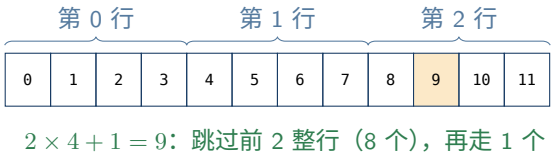
借用的外部情境　嵌入式系统中的内存映射寄存器。

为什么不算超纲　题干给出了“地址由芯片手册规定”“写 1 点亮 LED”“`volatile` 可忽略”等全部背景。

迁移点　笔记红字“空指针和野指针都不是我们申请的空间，因此不要访问”的反面：**只要空间是合法可访问的，按地址访问就是正当的**——判断标准始终是“这块内存是不是你的”。

第 38 题　跨学科　答案：9; 0x3009; 9; 0x3024　　考点：K1 K7 × 图像处理





解析图 38

推理　下标 $= r \times W + c = 2 \times 4 + 1 = 9$；`unsigned char` 每个 1 字节，地址 $=$ `0x3000` $+ 9 \times 1 =$ `0x3009`，写作 `*(img + 9)`。若用 `int` 存放，步长为 4：`0x3000` $+ 9 \times 4 =$ `0x3000` $+ 36 =$ `0x3024`（$36 = 0\text{x}24$）。

当前专题知识　数组连续存放、指针算术 $p + k$ 的地址公式（第一、六讲）。

借用的外部情境　图像处理中二维图像的“按行存储”。

为什么不算超纲　下标公式 $r \times W + c$ 和每像素字节数都已在题干给出。

迁移点　“地址 $=$ 首地址 $+ k \times$ `sizeof(T)`”这一关系被用于二维数据：先把二维位置换算成一维下标 k，再用同一个公式算地址。同样的 9，换成 `int` 后地址不同——再次说明“类型决定步长”。

高频易错点总表

	错误写法 / 想法	正确理解	见
1	`p` 的值就是 `a` 的值	`p` 存地址，`*p` 才是 `a`	第二讲，1
2	`int* p, q;` 定义了两个指针	只有 `p` 是指针，`q` 是 `int`	第二讲
3	`double*` 占 8 字节是因为 `double` 占 8 字节	指针大小只由地址宽度决定	第三讲，4
4	`sizeof(p)` 与 `sizeof(*p)` 相同	前者是指针大小，后者是所指对象大小	第三讲
5	空指针解引用会得到 0 / 一定报段错误	未定义行为，不得访问；应先判断	第四讲，5，12
6	程序没崩溃就说明没越界	越界可能“看起来正常”	第四、六讲
7	只记“常量指针/指针常量”的名字	看 const 在 * 左（锁值）还是右（锁指向）	第五讲，7，8
8	`const int *p` 使 `a` 成为常量	只是不能*通过* p 修改	第五讲，22
9	`p++` 使地址加 1	加 `sizeof(T)` 字节	第六讲，9，19
10	`*p+2` 等于 `*(p+2)`	* 优先于 +，前者是值加 2	第六讲，14
11	遍历次数随手写（笔记 `i<10`）	用 `sizeof(arr)/sizeof(arr[0])`	纠错 ①，24
12	传了指针，函数改 `p` 就能改外部	只有改 `*p` 才影响外部	第七讲，27
13	在函数内用 `sizeof(arr)` 求长度	形参是指针，结果是 8	第八讲, 15, 29
14	冒泡内层写 `j < n-j-1`	应为 `j < n-i-1`	纠错 ②, 30, 35

基础解题步骤总结

四类题的通用做法

读程序写结果（& 与 *）

画出每个变量的方框 → 逐行执行：遇到 `p = ...` 改箭头，遇到 `*p = ...` 改箭头终点的值 → 最后按图读出输出。

判断 const 语句是否合法

找 * → 看 const 在左/右 → 看语句改的是 *p 还是 p → 对应一侧有锁则非法。“读”永远合法。

数组与指针计算

先确定元素类型大小 s → 地址 $(p+k)$ = 地址 $(p)+k\cdot s$ → 注意十六进制加法 → 检查是否越界。

函数能否修改实参

看形参是不是指针 → 是指针时再看函数改的是 *p（影响外部）还是 p（不影响）。

冒泡排序过程

第 i 轮比较 $j=0\sim n-i-2$ → 每比较一对写一次状态 → 每轮末检查“最大者是否到了第 $n-1-i$ 位”。

相近概念总对比

概念 A / 概念 B	本质区别	判断方法
p / *p	地址 / 对象	箭头起点 / 箭头终点
&a / &p	两个不同变量的地址	看方框上方的小字
声明中的 * / 表达式中的 *	类型的一部分 / 解引用	是否跟在类型名后面
空指针 / 野指针	确定的“无指向” / 非法的指向	能否用 == nullptr 检查
const int *p / int * const p	锁值 / 锁指向	const 在 * 左 / 右
p + 1 / 整数 + 1	走一个元素 / 加 1	看左边是不是指针
sizeof(arr) 在 main 中 / 在函数中	整个数组 / 一个指针	看 arr 是数组还是形参
值传递 / 地址传递	拷贝值 / 拷贝地址	形参是否为指针，函数是否改 *p

后续课程连接

> **本章知识以后在哪里还会出现**
>
> - **C++ 后续章节**：引用（`int &`，更安全的“别名”）、结构体指针与 `->`（C 组第 36 题已接触）、动态内存 `new/delete`（指针指向“没有名字”的内存）、智能指针。
> - **数据结构**：链表、树、图的结点通过指针连接；本章的“遍历”“越界”“空指针检查”会反复出现。
> - **计算机组成原理**：地址宽度与寻址空间（第三讲的 2^{32} 字节）、内存按字节编址。
> - **操作系统**：虚拟内存与内存保护——段错误的真正来源（第四讲）。
> - **嵌入式系统**：内存映射寄存器（C 组第 37 题）。
> - **算法设计与分析**：冒泡排序的 $n(n-1)/2$ 次比较是“时间复杂度”这一概念的第一个例子（图 8.3）。

参考资料

原始教学依据

- 课堂手写笔记，C++ 第七章“指针”，笔记页 378–393，2026 年 9 月 25–27 日（本讲义的专题边界与全部原始内容来源）。

核心定义与语言规则核验

- ISO/IEC 14882，*Programming languages — C++*（C++ 国际标准；下标运算 `E1[E2]` 定义为 `*((E1)+(E2))`、数组到指针的转换、未定义行为等条款）。
- cppreference.com：*Pointer declaration*、*nullptr, the pointer literal (since C++11)*、*cv (const and volatile) type qualifiers*、*Array-to-pointer conversion*、*sizeof operator*。https://en.cppreference.com/
- Stanley B. Lippman, Josée Lajoie, Barbara E. Moo. *C++ Primer*（第 5 版），第 2.3.2 节“指针”、第 2.4.2 节“指针和 const”、第 3.5.3 节“指针和数组”。
- 实测环境：g++（C++17），x86-64 Linux；讲义中全部程序输出、编译错误信息与 38 道练习的答案均在此环境中实际编译运行核对。

学科史与科普资料

- Dennis M. Ritchie. *The Development of the C Language.* ACM HOPL-II, 1993.（BCPL、B 到 C 的演变；数组名转换为指针的设计动机。）
- Bjarne Stroustrup. *A History of C++: 1979–1991.* ACM HOPL-II, 1993.（C with Classes 始于 1979 年，1983 年改名 C++。）
- Bjarne Stroustrup. *C++11 FAQ*（`nullptr` 的引入）。

- Tony Hoare. *Null References: The Billion Dollar Mistake*. QCon London 演讲，2009.（1965 年在 ALGOL W 中引入空引用的回顾。）